ALGORITHM ANALYSIS AND DESIGN (AAD)

Algo
DS

Problem → Search element

4	8	6	7	10	12

Key = 7

Linear search = O(n)

Binary search = O(logn)

n = 32

logn < n

$\log_2 2^5 < 32$

1 5 < 32

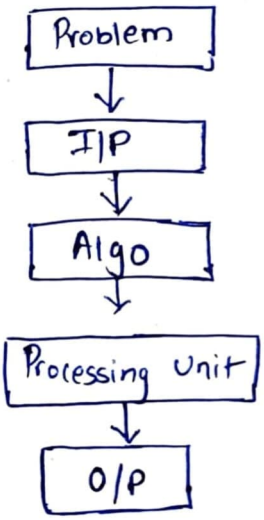
Algorithm

An algorithm is a finite set of instruction that follow a particular task.

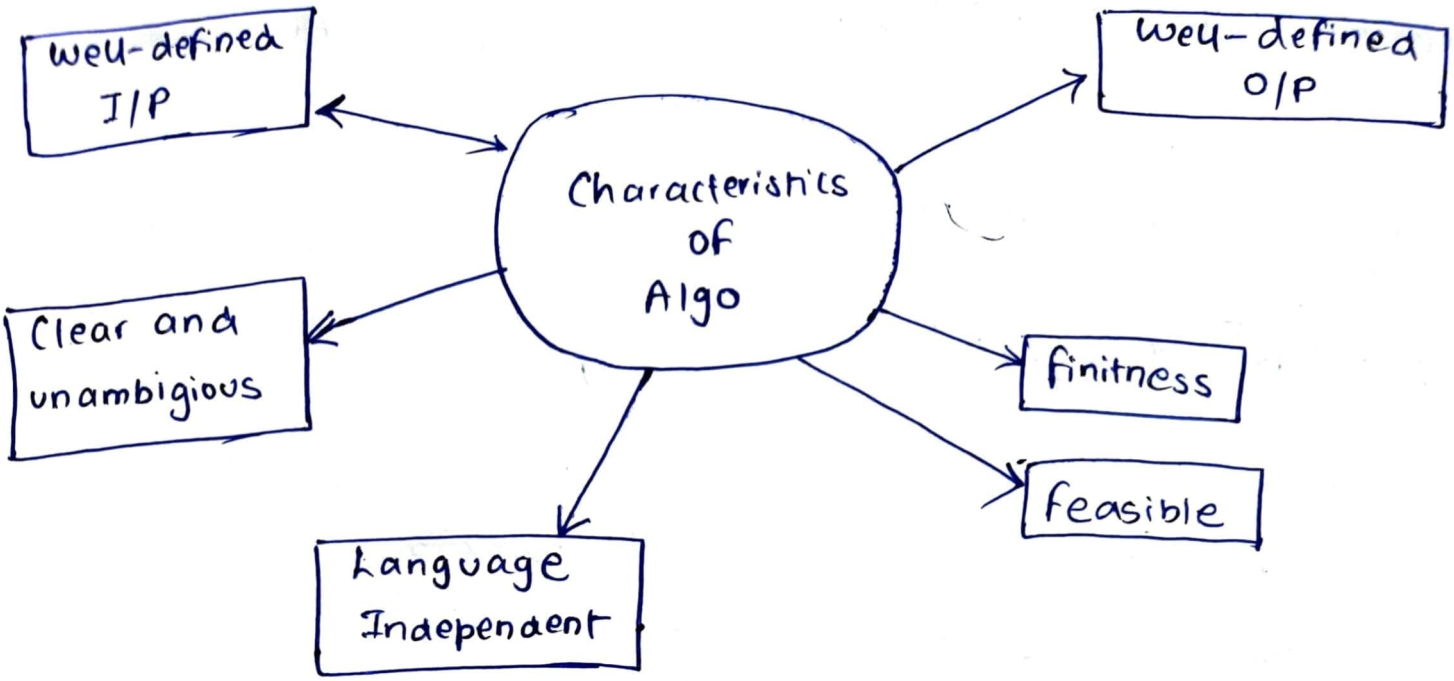
All algorithm must satisfy the following criteria :→

→ Input
→ output
→ Definitness
→ Finitness
→ Effectiveness

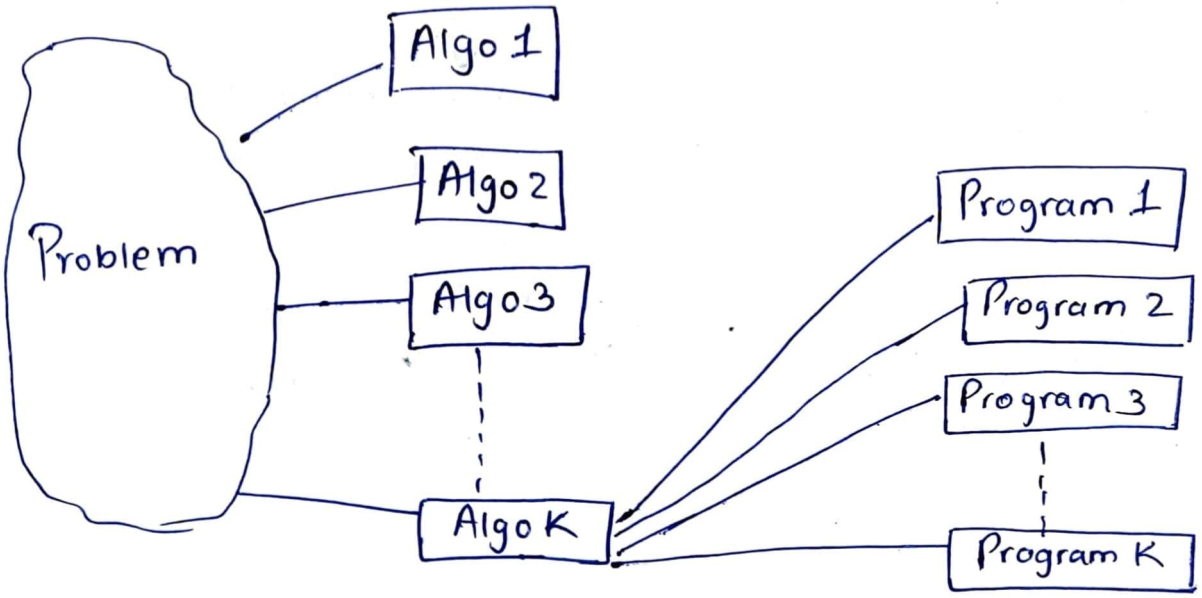
Data flow of Algorithm



(Abhyudaya Srivastava)



ALGO Vs PROGRAM



CLASSIFICATION OF ALGO

→ Reursive Algo

→ Backtracking Algo

→ Divide and conquer Algo

→ Dynamic programming Algo

→ Greedy Algo

→ Branch and bound Algo

→ Bruteforce Algo

→ Randomised Algo

Algo- Analysis

→ Input size

→ Running time (Worst Case + Average Case)

Linear Search =

0	1	2	3	-	--
5	7	8	6	7	

Worst case = $O(n)$

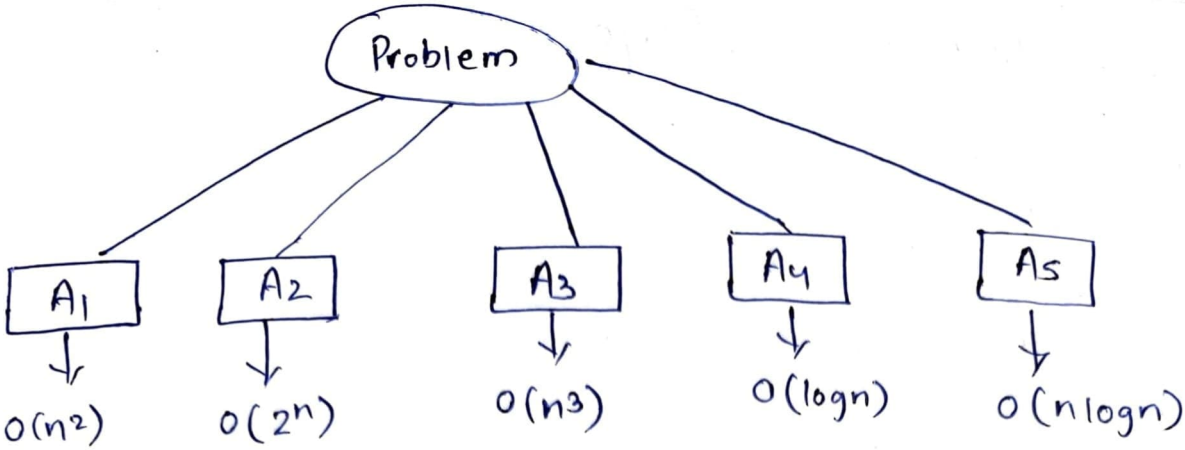
Best case = $O(1)$

$$A.C = \frac{1+2+3+4+---+n}{n}$$

$$= \frac{\frac{n(n+1)}{2}}{n} = \frac{(n+1)}{2} = O(n)$$

→ Order of growth function

for Example :→ $5n^2+3n+2 = O(n^2)$

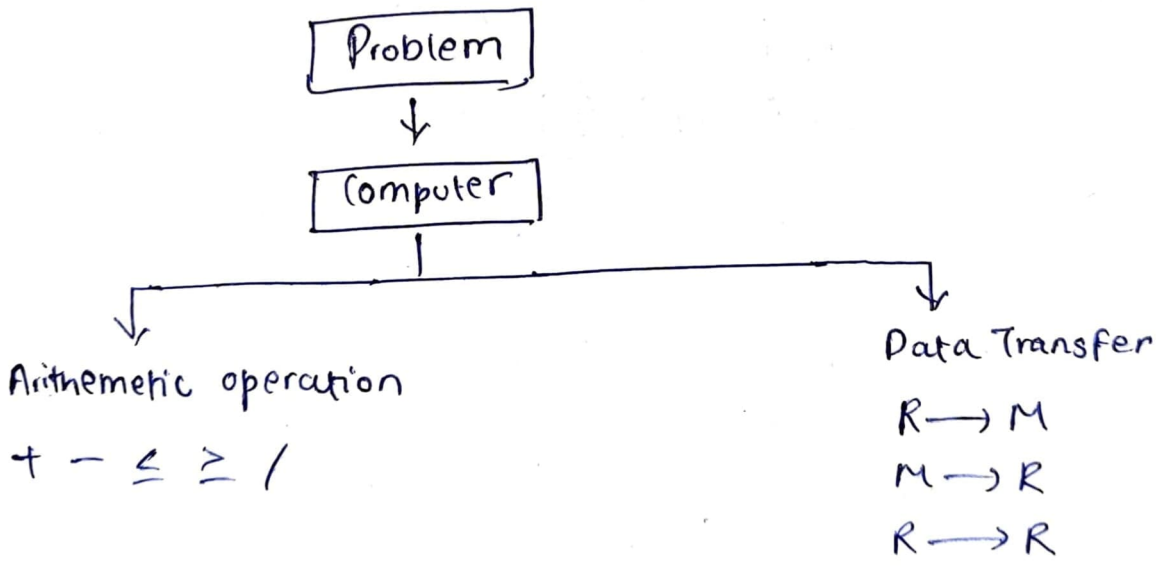


Types of complexity

→ Time complexity

→ Space complexity

The count of no. of primitive operation (stack) as a function of input size. Primitive operation done with two methods



Space complexity defined as memory storage and it depends upon input size.

Addition Of Two Matrix

```
Sum (A, B, n)                          when matrix = 3x3      when matrix = nxn
for (i=0, i<=2 ; i++)  ———————————  O(3)  ———————————  O(n²)
 {
   for (j=0 ; j<=2; j++)  ———————  O(3)  ———————————  O(n)
     {                                =O(1)              => O(n²)
       print [i][j]
     }
 }
```

Time complexity = $O(n^2)$

space complexity = $O(n^2)$

Space complexity

$\Rightarrow O(n^2) + O(n^2) + O(n^2)$

$\Rightarrow 3(n^2)$

$\Rightarrow O(n^2)$

Frequency Count Method

```
Algo sum (A, n)
  {
   sum = 0
   for ( i=0; i<n; i++)  —— O(n+1)
     {
      Sum = Sum + A[i]  —— O(n)
     }
     Return Sum  —— O(1) —— O(1)
  }
```

$T(n) = 1 + n + 1 + n + 1$

$= 2n + 3$

$\Rightarrow O(n)$

(I)
```
main()
{
  int x,y,z;  ---- 1
  x=y+z;      ---- 1
}
```

$T(n) = 1+1$
$= O(1)$

(II)
```
main()
{
  int n,i;
  for(i=0; i<=n; i++)  ---- n+1
  {
    statement  ---- n
  }
}
```

$T(n) = n+1+n$
$= 2n+1$
$= O(n)$

```
Algo add(A, B, n)
{
  for(i=0; i<n; i++)  ---- n+1
  {
    for(j=0; j<n; j++)  ---- n(n+1)
    {
      C[i,j] = A[i,j] + B[i,j]  ---- n x n
    }
  }
}
```

$T(n) = n+1+n^2+n+n^2$
$= 2n^2+2n+1$
$= O(n^2)$

variable space
A → n^2
B → n^2
C → n^2 ⇒ $3n^2+3$ ⇒ $O(n^2)$
i → 1
j → 1
n → 1

Algo multiply (A, B, n)

```
for(i=0; i<n; i++)  — n+1
  {
   for(j=0; j<n; j++)  — n(n+1)
   {
    C[i,j]=0 — n×n
    for(k=0; k<n; k++) — n×n(n+1)
    {
      C[i,j]=C[i,j] + A[i,k] * B[k,j] — n×n×n
    }
   }
  }
}
```

$T(n) = n+1+n^2+n+n^2+n3+n2+n3$

$\Rightarrow 2n3 + 3n^2 + 1$

$\Rightarrow O(n3)$ Ans

$S(n) = A + B + C + i + j + K + n$

$= n^2 + n^2 + n^2 + 1 + 1 + 1 + 1$

$= O(n^2)$ Ans

```
main()
{
 int i, n;
 for(i=1; i<=n; i=i*2)
  {
    statement
  }
}
```

$i = 1+2+4+8+16+\cdots 2^K$

$2^K = n$

$K = O(\log_2 n)$

```
main()
{
 int i, n;
 for (int i=n; i >= 1 ; i = i/2)
  {
    statement
  }
}
```

$n/2 = n/4 = n/8 \cdots\cdots \frac{n}{2^K}$

$\frac{n}{2^K} = 1$

$n = 2^K$

$K = \log_2 n$

$\Rightarrow O(\log_2 n)$

```
main()
{
  int i, j, n;  ---- 1
  for (i=1; i<=n; i++)  ---- n+1
    {
      for(j=1; j<=n; j=j*2)  ---- n logn
      {
        printf("GLA")
      }                          => O(n logn)
    }
}
```

```
main()
{
  int p, i, j, n;
  p=0
  for (i=1; i<n; i=ix2)
  {
    p++;  ---- logn
  }
  for( j=1; j<p; j=j*2)
    {
      statement ---- log p        => O(log(logn))
    }
}
```

Comparison Function (growth function)

formula of log

1. $\log(m \times n) = \log m + \log n$
2. $\log\left(\frac{m}{n}\right) = \log m - \log n$
3. $\log_a a = 1$

→ $\log_n m$

→ $m \log n$

→ $a \log_b n = n \log_b a$

Types of Time Functions

1. $O(1)$ → Constant Time

$f(n) = 3$
$f(n) = 100$ — $O(1)$
$f(n) = 1000$

2. $O(\log_2 n)$ → logrithemic time

3. $O(n)$ → Linear time

$f(n) = 2n + 3$
$f(n) = 5n - 6$ — $O(n)$
$f(n) = \frac{n}{1000} + 5$

4. $O(n^2)$ → Quadratic time ← $f(n) = 2n^2 + 2n + 2$, $f(n) = \frac{n^2}{100} + 5n + 2$, $f(n) = \frac{n2}{2} - 6n + 1$ → $O(n2)$

5. $O(n3)$ → Cubic Time ← $f(n) = 3n^3 + 5n^2 + 2$, $f(n) = \frac{n^3}{100} + 3n + 1$ → $O(n3)$

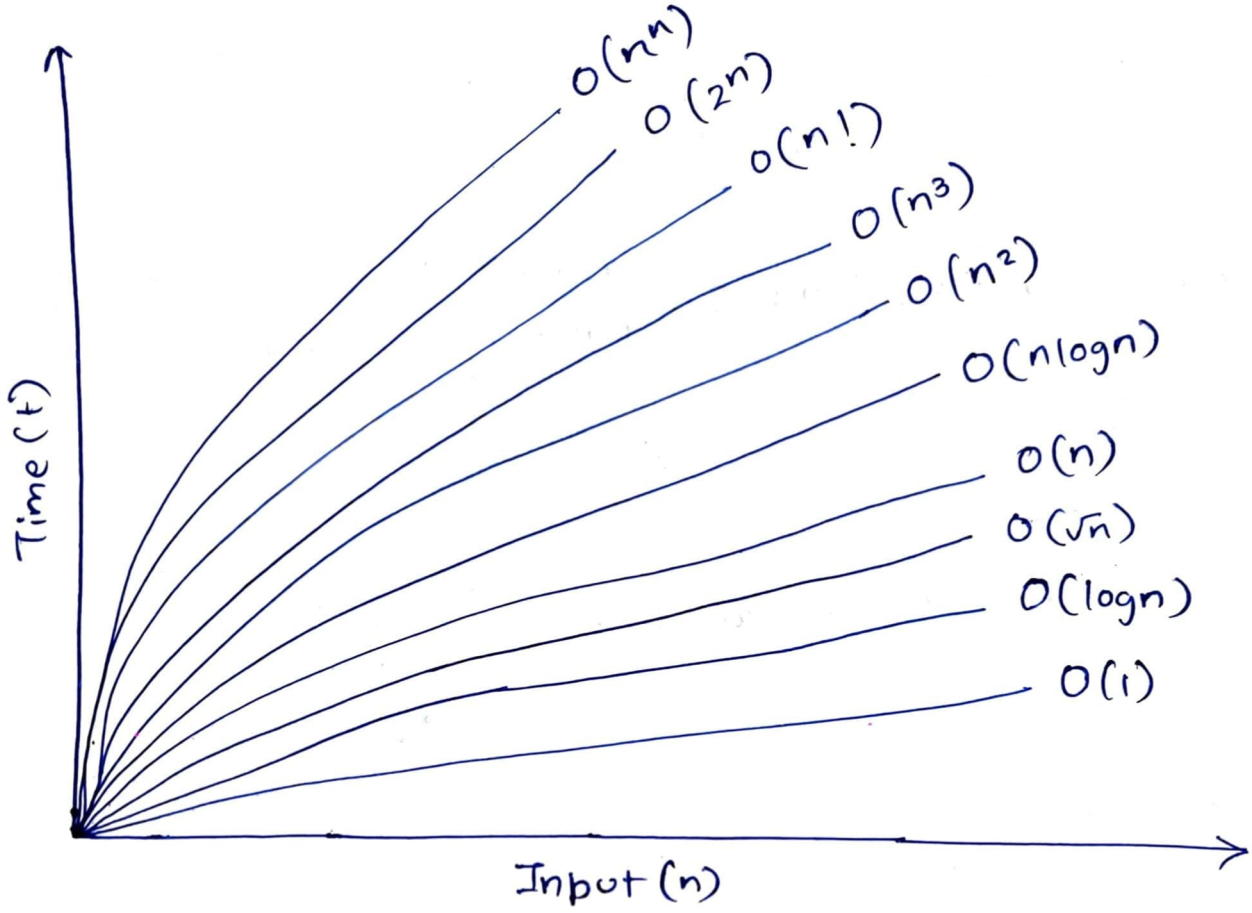
6. $O(2^n)$ ⟶ Exponential Time

$O(3^n)$

$O(n^n)$ ....

$1 < \log n < n < n^2 < n^3 < 2^n$ ---- $< n^n$

	$\log_2 n$	n	n^2	2^n
$n=1$	0	1	1	2
$n=2$	1	2	4	4
$n=3$	13	3	9	8
$n=4$	2	4	16	16
$n=5$	2.5	5	25	32



(Approximate)

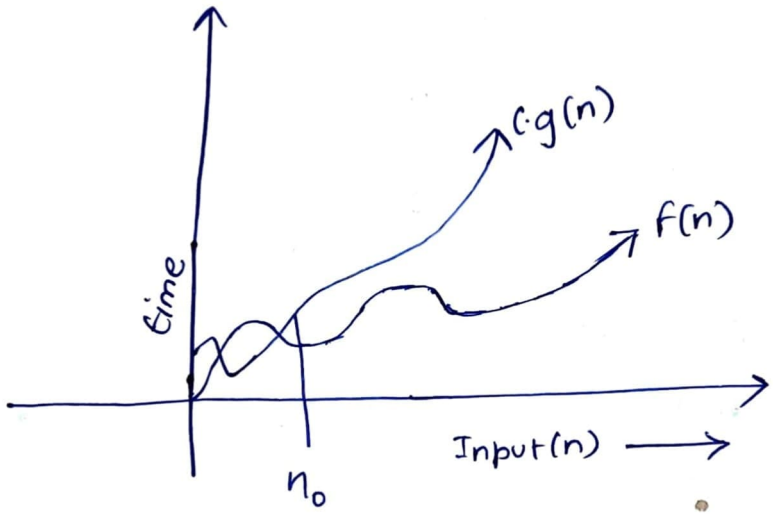
ASYMPTOTIC NOTATION

- Big-Oh Notation (O) → upper bound
- Big-Omega Notation (Ω) → lower bound
- Theta Notation (θ) → Average bound

Big-oh Notation (O)

$f(n) = O(g(n))$ iff

$f(n) \leq c \cdot g(n)$ for some value of $c > 70$ & for all $n \geq 0 n_0$



	$f(x) = n^2$	$g(n) = 2^n$
n=1	1	2
n=2	4	4
n=3	9	8
n=4	16	16
n=5	25	32
n=6	36	64

$f(n) \leq c\ g(n)$

$n^2 \leq c \cdot 2^n$

$16 \leq c \cdot 16$

$c = 1$

$f(n) = O(2^n)$

$f(n) = 3n+2$ $\quad g(n) = n$

	$f(n) = 3n+2$	$g(n) = n$
$n=1$	5	1
$n=2$	8	2
$n=3$	11	3
$n=4$	14	4
$n=5$	17	5

$3n+2 \leq cn$

$3\times1+2 \leq 5\times1$

$5 \leq 5$

$f(n) = 2n^2+3n+5$

$g(n) = n^2$

$2n^2+3n^2+5n^2 \leq (10)n^2$

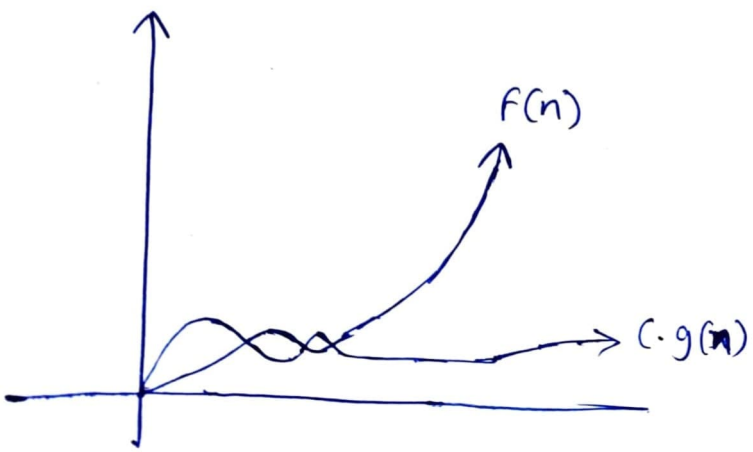
c

Big - Omega Notation (Ω)

$f(n) \geq c \cdot g(n)$

for some value of $c>0$ && for all $n \geq n_0$

then $f(n) = \Omega\, g(n)$



$f(n) = 3n+2$ $\quad g(n) = n$

	$f(n) = 3n+2$	$g(n) = n$
$n=1$	5	1
$n=2$	8	2
$n=3$	11	3
$n=4$	14	4

$f(n) \geq c \cdot g(n)$

$3n+2 \geq c \cdot n$

$5 \geq 1$

$c=1$
$c=2$
$c=3$
$c=4$
$c=5$

Theta Notation (θ)

The functions $f(n) = \theta\, g(n)$ iff

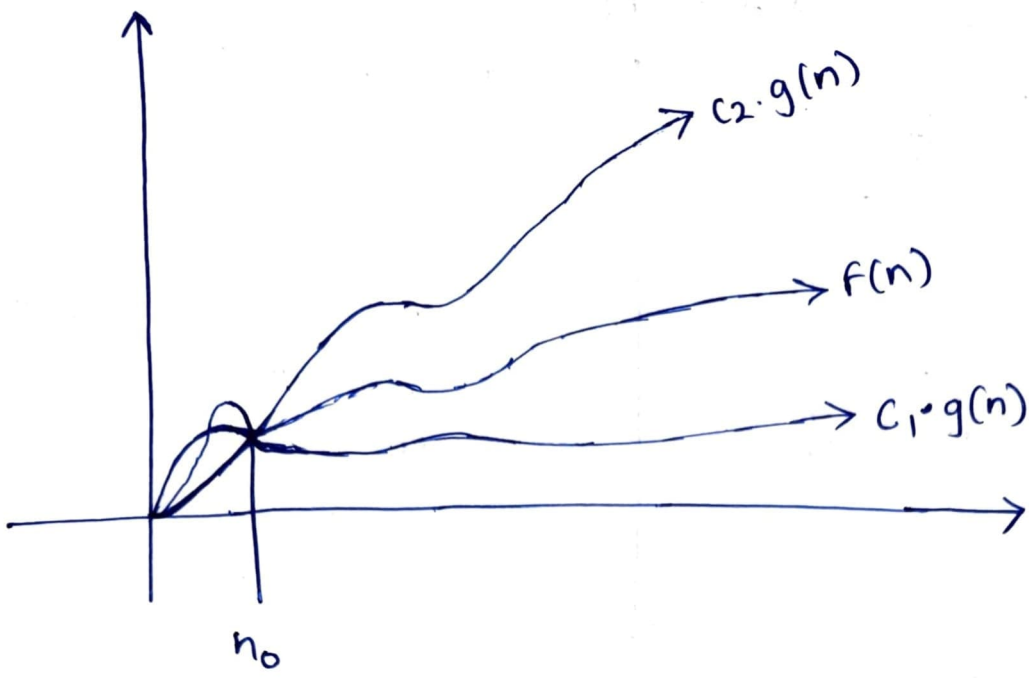
Some positive constant c_1, c_2 and no

Such that

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

$\Omega \quad + \quad O$

$= \theta$



$f(n) = 2n+3$ $\quad$ $g(n) = n$

$1*n \leq$	$2n+3 \leq$	$5 \times n$
Ω	θ	

$c_1 = 1$
$c_2 = 5$
$n_0 = 1$

$f(n) = \theta(g(n))$

Small O Notation

$f(n) = o(g(n))$, iff $f(n) < c\,g(n)$

for all values of $c > 0$ && all value of $n \geq n_0$

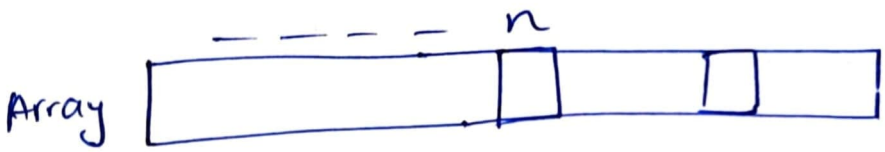
Small Omega Notation (ω)

$f(n) = \omega(g(n))$ iff

$f(n) > c\,g(n)$ for all value of $c > 0$ && for all value of $n \geq n_0$

Recurrences Method

- Substitution method
- Iteration method (Back Substitution, Summation method)
- Recursion tree method
- Master theorum method



BS (A, n, key)

{

while (l≤r)

{

mid = $\frac{l+r}{2}$ → C

l = mid + 1 — n/2

also

r = mid - 1 — n/2

n/2, n/4, n/8 - - - -

T(n) = T(n/2) + C

$T(n) = T(n/2) + c$

$T(n/2) = T(n/4) + c$

$T(n) = T(n/4) + c + c$

$T(n/4) = T(n/8) + c$

$T(n) = T(n/8) + c + c + c$

$T(n/8) = T(n/16) + c$

$T(n) = T(n/16) + c + c + c + c$

$\vdots$

$T(n) = T(n/2^k) + kc$

Also $n/2^k = 1$

$n = 2^k$

$\log n = \log_2 2^k$

$\log n = k \log_2 2$

$\log n = k \cdot 1$

$\boxed{k = \log n}$

The Substitution Method

$T(n) = T(n/2) + 1$

we have to show that it is asymptotically bound by $O(\log n)$

$T(n) = O(\log n)$

$T(n) \leq c \cdot \log n \leq c \cdot \log(n/2) + 1 \leq c[\log n - \log_2 2] + 1$

$\Rightarrow c \cdot \log n - c \cdot \log_2 2 + 1$

$\log \frac{m}{n} = \log m - \log n$

$\Rightarrow c \cdot \log n - c + 1$

$T(n) = O(\log n)$

Q Consider the recurssion

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

find the asymptotic bound on $T(n)$

Sol: we guess the solution is $O(n \log n)$

thus for a constant c.

$T(n) \leq c\, n \log n$

$\Rightarrow \leq 2c\,(n/2)\log(n/2) + n$

$\Rightarrow \leq cn\log n - cn\log_2 2 + n$

$\log_2 2 = 1$

$\Rightarrow \leq cn\log n - cn + n$

$T(n) = O(n\log n)$

Q $T(n) = \begin{cases} 1 & , n=1 \\ T(n-1)+C, & \text{where } n>1 \end{cases}$ using Iterative method

Sol. $T(n) = T(n-1)+C$

$T(n-1) = T(n-2)+C$

$T(n) = T(n-2)+C+C$

$T(n-2) = T(n-3)+C$

$T(n) = T(n-3)+C+C+C$

$T(n-3) = T(n-4)+C$

$T(n) = T(n-4)+C+C+C+C$

⋮ k times

$T(n) = T(n-k)+kC$

$n-k=1$

$k = n-1$

$T(n) = T(n-n+1)+C(n-1)$

$T(n) = T(1)+nC-C$

$= 1+nC-C$

$T(n) = O(n)$

Q Solve using Iterative method (Iteration method)

$$T(n) = \begin{cases} 1 & , n=1 \\ 2T(n/2)+n & , \text{when } n>1 \end{cases}$$

Sol. $T(n) = 2T(n/2) + n$

$T(n/2) = 2T(n/4) + n/2$

$T(n) = 2(2T(n/4) + n/2) + n$

$= 2^2 T(n/4) + 2 \times n/2 + n$

$T(n) = 2^2 T(n/4) + n + n$

$T(n/4) = 2T(n/8) + n/4$

$T(n) = 2^2 \left(2T(n/8) + \frac{n}{4}\right) n + n$

$T(n) = 2^3 T(n/8) + 4 \times \frac{n}{4}) + n + n$

$T(n) = 2^3 T(n/8) + 3n$

⋮ K time

$T(n) = 2^K T(n/2^K) + kn$

$\frac{n}{2^K} = 1$

$n = 2^K \Rightarrow K = \log_2 n$

$2^{\log_2 n}(1) + n \log_2 n$

$\Rightarrow n \log_2 2 (1) + n \log_2 n$

$\Rightarrow n \times 1 + n \log n$

$= O(n \log_2 n)$

Q $T(n) = \begin{cases} 1 & , n=1 \\ T(n-1)+n & , \text{when } n \geq 1 \end{cases}$

Sol. $T(n) = T(n-1)+n$

$T(n-1) = T(n-2)+n-1$

$T(n) = T(n-2)+n-1+n$

$T(n-2) = T(n-3)+n-2$

$T(n) = T(n-3) + T(n-2) + n-1+n$

: k time

$T(n) = T(n-k) + (n-(k-1)) + (n-(k-2))$

$n-k=0$

$k=n$

$T(n) = T(n-n) + (n-(n+1)) + (n-(n-2)) + (n-(n-3)) + n$

$T(n) = T(0) + 1+2+3+ \dots +n$

$1+2+3+ \dots +n$

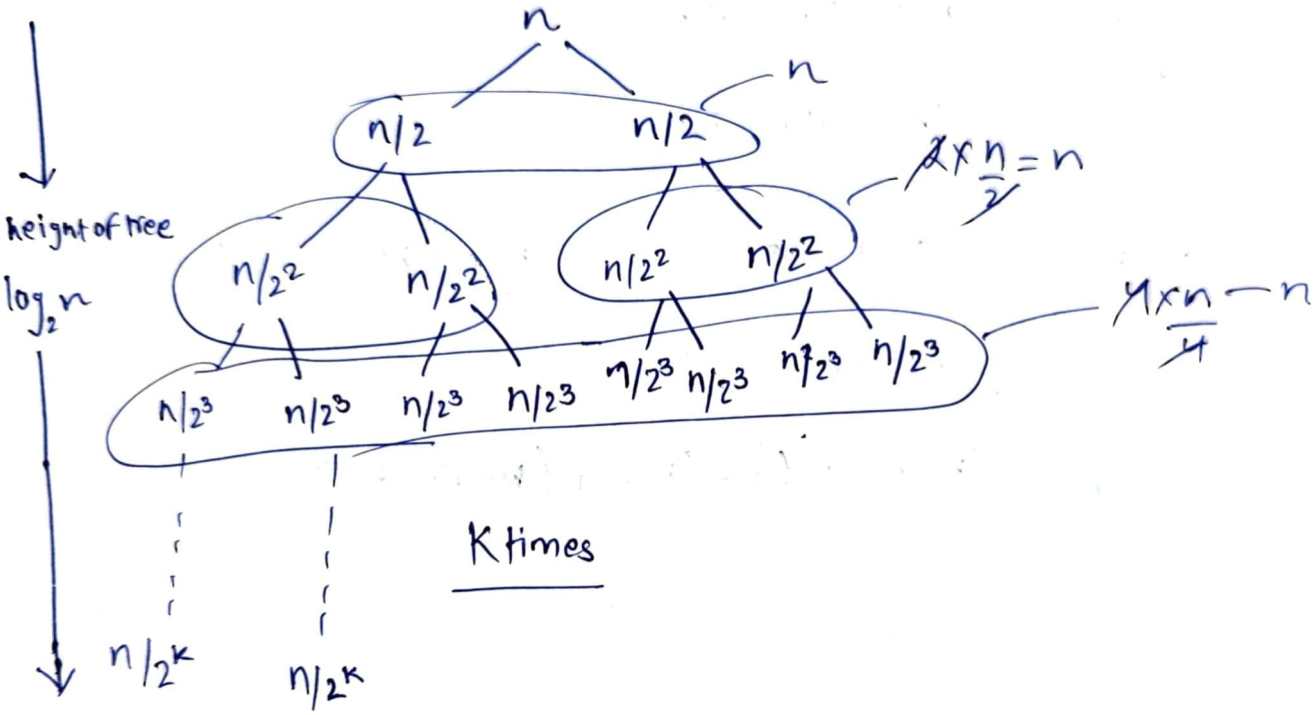
$\Rightarrow \underline{\frac{n(n+1)}{2}} = \underline{O(n^2)}$

Recursive tree Method

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 2T(n/2)+n & \end{cases}$$ — Merge sort

$$T(n) = 2T(n/2) + n$$

$$= T(n/2) + T(n/2) + n$$



$$\frac{n}{2^K} = 1$$

$$n = 2^K$$

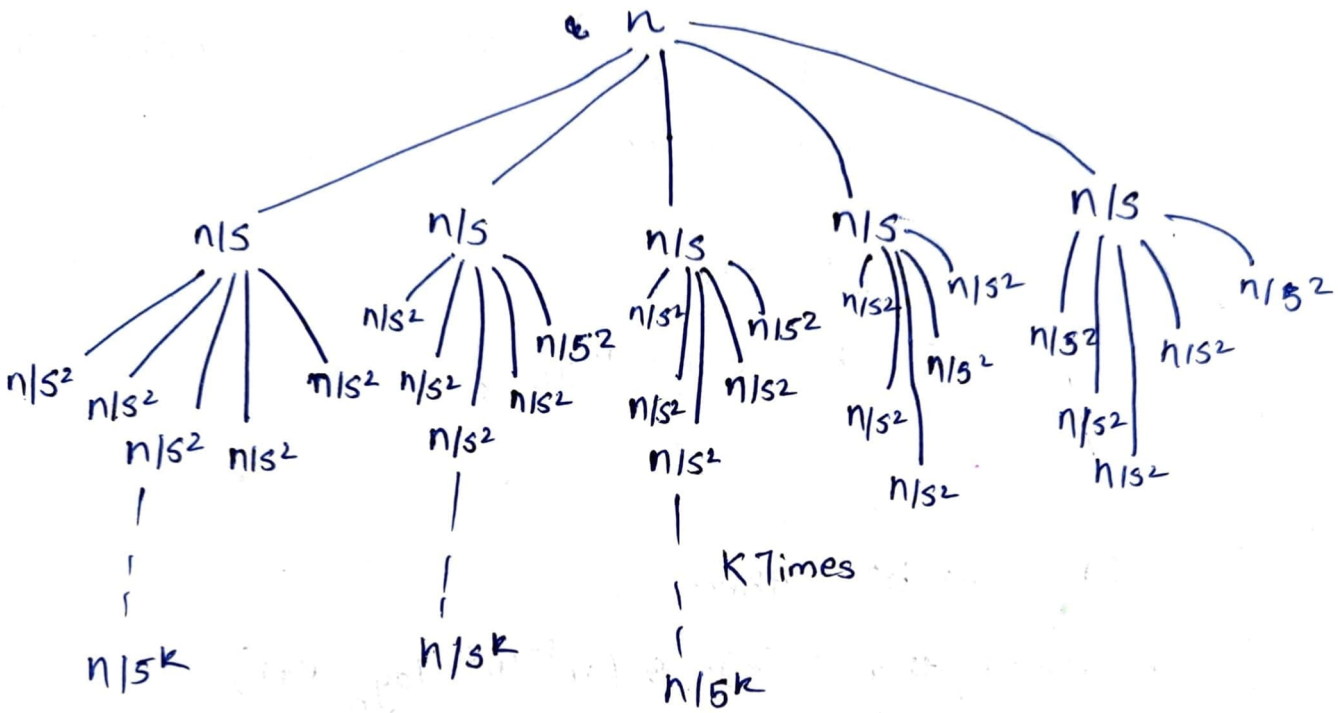
$$\boxed{K = \log_2 n}$$

$$\Rightarrow n+n+n+n \ldots + n$$

$$\Rightarrow kn$$

$$\Rightarrow O(n \log n)$$

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 5T(n/5) + n \end{cases}$$



$n/5^k = 1$

$k = \log_5 n$

$\Rightarrow n + n + n - - - nk$

$\Rightarrow kn$

$\Rightarrow O(n \log_5 n)$

$T(n) = T(n/3) + T(2n/3) + n$

$T(n/3) = T(n/3^2) + T\left(\frac{2n}{3^2}\right) + n/3$

$T\left(\frac{2n}{3}\right) \Rightarrow T\left(\frac{2n}{3^2}\right) + \left(\frac{4n}{3^2}\right) + \frac{2n}{3}$

Master Method

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \text{ with } a \geq 1 \text{ and } b \geq 1$$

$$T(n) = aT\left(\frac{n}{b}\right) + \theta\left(n^k \log_n^p\right) \quad \text{where } a \geq 1, b \geq 1, k \geq 0$$

and p is a real number

Case 1:→ If $a > b^k$ then

$$T(n) = \theta\left(n \log_b a\right)$$

Case 2:→ If $a = b^k$ then

(a) $p > -1$, then $T(n) = \theta\left(n^{\log_b a} \log_n^{p+1}\right)$

(b) $p = -1$, then $T(n) = \theta\left(n^{\log_b a} \times \log\log n\right)$

(c) $p < -1$, then $T(n) = \theta\left(n^{\log_b a}\right)$

Case 3:→ If $(a < b^k)$

(a) If $p \geq 0$ then $T(n) = \theta\left(n^k \log_n^p\right)$

(b) If $p < 0$ then $T(n) = \theta(n^k)$

Q $T(n) = 4T(n/2) + n$

$= 4T(n/2) + n^1 \log^0 n$

$a=4, b=2, k=1, p=0$

$(a > b^k) = 4 > 2^1 \longrightarrow$ case.

$T(n) = \Theta(n^{\log_b a}) \Rightarrow n^{\log_2 4} \Rightarrow n^{\log_2 2^2}$

$= \Theta(n^2)$

Q $T(n) = 2T\left(\frac{n}{2}\right) + n^1 \log^0 n$

$a=2, b=2, k=1, p=0$

$a = b^k \Rightarrow 2 = 2^1 = (a = b^k) \longrightarrow$ case 2

$p > -1 \Rightarrow T(n) \Rightarrow \Theta(n^{\log_b a} \log_n^{p+1})$

$= (n^{\log_2 2} \times \log_n^{0+1})$ ($\log_2 2 \to 1$)

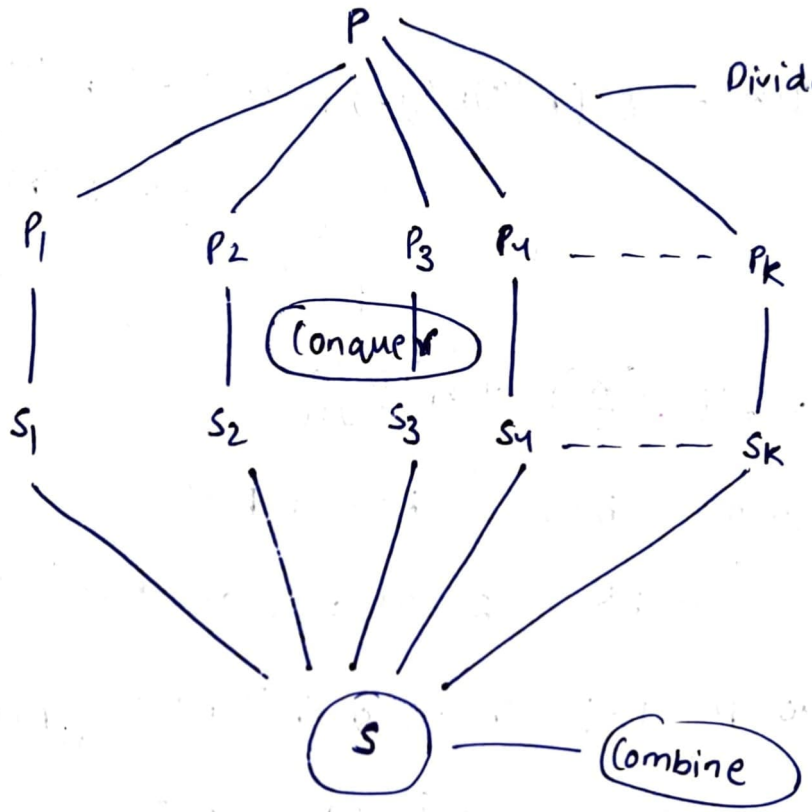
$= (n \log n)$

Sorting Techniques

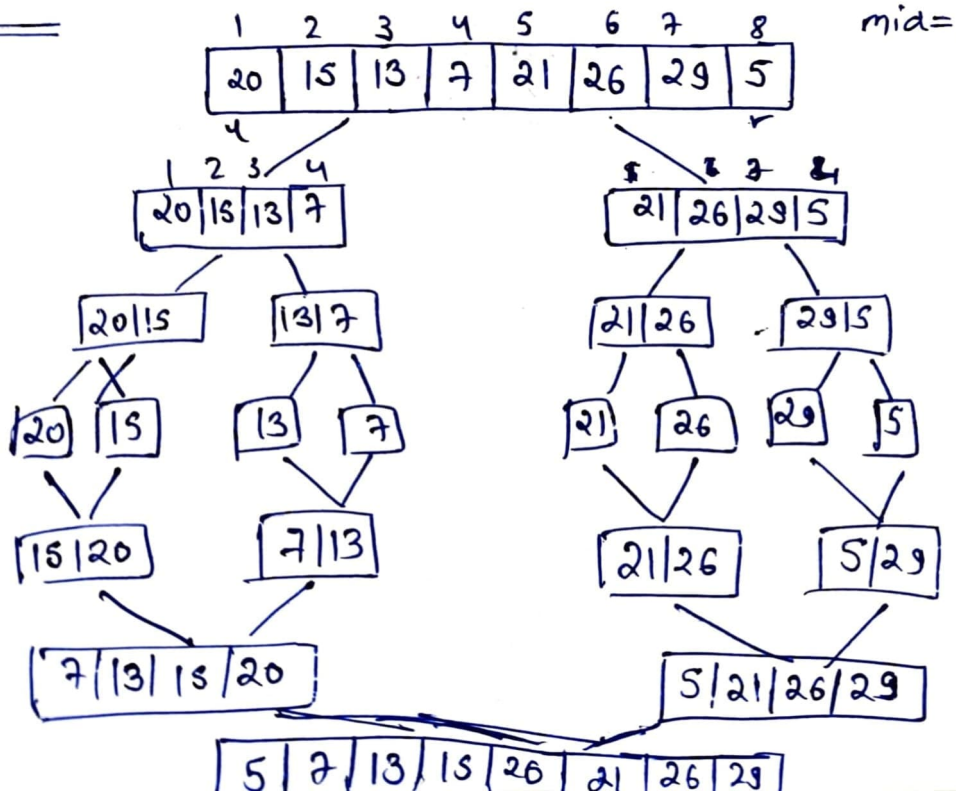
Ex:→
- Merge sort
- Quick sort

→ Divide and conquer

- Divide
- Conquer
- Combine



Merge Sort



Merge (A, p, q, r)

1. $n_1 = q - p + 1$
2. $n_2 = r - 1$
3. Let $L[1 \ldots n_1+1]$ and $R[1 \ldots n_2+1]$ // New Array
4. For ($i = 1$ to n_1)
5. $L[i] = A[p+i-1]$
6. For ($j = 1$ to n_2)
7. $R[j] = A[q+j]$
8. $L[n_1+1] = \infty$
9. $R[n_2+1] = \infty$
10. $i = 1$, $j = 1$
11. For ($K = p$ to r)
12. if ($L[i] \leq R[j]$)
13. $A[k] = L[i]$
14. $i = i+1$
15. else $A[k] = R[j]$
16. $j = j+1$

$q = \frac{p+r}{2}$

$q = 4$

	1	2	3	4	5	6	7	8
A =	10	20	30	40	11	21	31	41
	p			q				r

Time Complexity => $n+n+n = 3n = O(n)$

Space Complexity => $2n + 2(\infty) = O(n)$

Merge Sort (A, P, r)

IF (P < r)

$q = \left\lfloor \frac{p+q}{2} \right\rfloor$

Merge sort (A, P, q) ← n/2

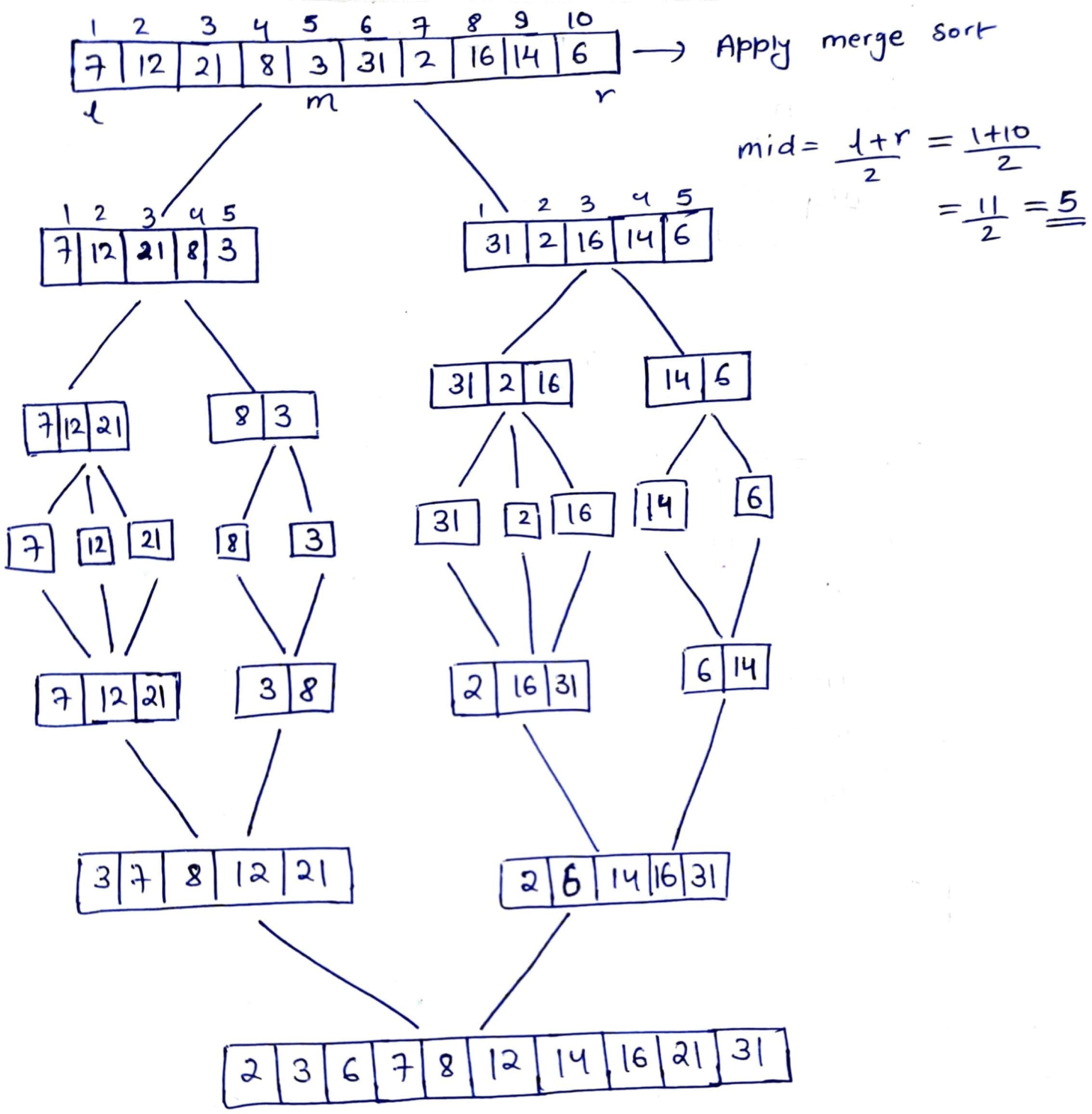
Merge sort (A, q+1, r) ← n/2

Merge Sort (A, P, q, r) ← O(n)

$T(n) = 2T(n/2) + n$

$= O(n \log n)$

→ Suppose if Array size is very large then Merge sort is the technique to solve it.


1 2 3 4 5 6 7 8 9 10
7 12 21 8 3 31 2 16 14 6
l
m
r
→ Apply merge sort
mid = (l+r)/2 = (1+10)/2
= 11/2 = 5
1 2 3 4 5
7 12 21 8 3
1 2 3 4 5
31 2 16 14 6
7 12 21
8 3
31 2 16
14 6
7
12
21
8
3
31
2
16
14
6
7 12 21
3 8
2 16 31
6 14
3 7 8 12 21
2 6 14 16 31
2 3 6 7 8 12 14 16 21 31

Q $f(n) = n+10$ find out c and n_0

$g(n) = n-10$

$\boxed{f(n) = O(g(n))}$

Sol. $f(n) = O(g(n))$

$f(n) \leq c \cdot g(n)$

$n+10 \leq 2(n-10)$ $\quad c=2$

$n+10 \leq 2n-20$ $\quad n_0 = 30$

$30+10 \leq 2 \times 30 - 20$

$40 \leq 60-20$

$40 \leq 40$

$\boxed{c=2 \quad n_0 = 30}$

Q $$T(n) = \begin{cases} 1 & n=1 \\ 4T(n/2)+n & \text{when } n>1 \end{cases}$$

Solving using ① Master Method ② Iterative method

Sol. ② Iterative method

$T(n) = 4T(n/2)+n$

$T(n/2) = 4T(n/4) + n/2$

$T(n) = 4(4T(n/4) + n/2) + n$

$= 4^2 T(n/4) + 2(n/2) + n$

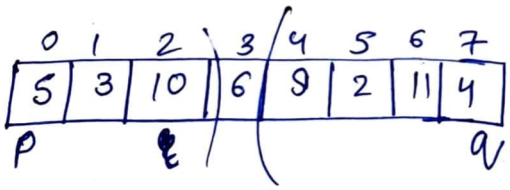
Quick Sort

→ Choose a pivot element

→ partition of array

→ Recursively sort the sub-array

```
partition (A, P, q)
{
    x = A[P]
    i = P
    for (j = P+1 ; j ≤ q ; j++)
    {
        if (A[j] ≤ x)
        {
            i = i+1
            swap (A[i], A[j])
        }
    }
    swap (A[i], A[P])
}
```

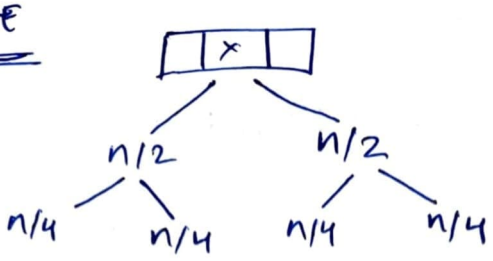


```
Quick Sort (A, P, q)
{ if (P < q)
} on { partition (A, P, q)
    Quick Sort (A, P, m-1)
    Quick Sort (A, m+1, q)
```

$T(n) = O(n \log_2 n)$

$T(n) = T(n-1) + n \Rightarrow O(n^2)$

Best case:



Best case $T(n) = O(n \log n)$

Worst Case :→

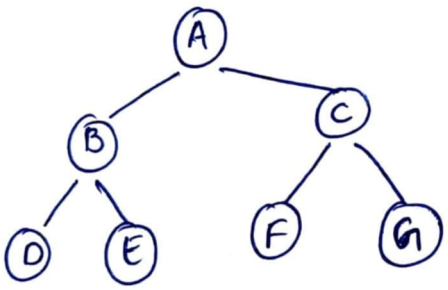
Both sides not equal

$T(n) = T(n-1) + n$

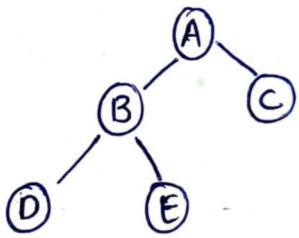
$T(n) = O(n^2)$

Heap Tree

Complete Binary Tree:



Almost Complete Binary Tree:→



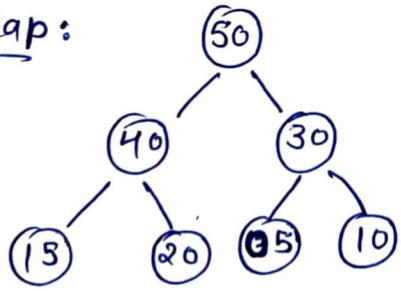
1	2	3	4	5	6	7
A	B	C	D	E	F	G

Left Child: $2i$

Right Child: $2i+1$

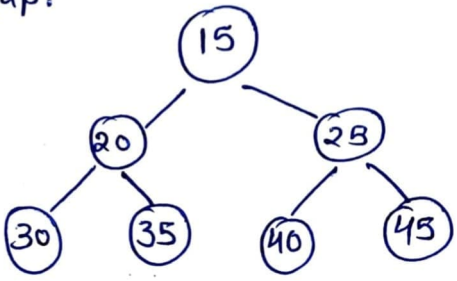
Parent (i): $\left[\frac{i}{2}\right]$

1. Max Heap:



1	2	3	4	5	6	7
50	40	30	15	20	05	10

2. Min Heap:



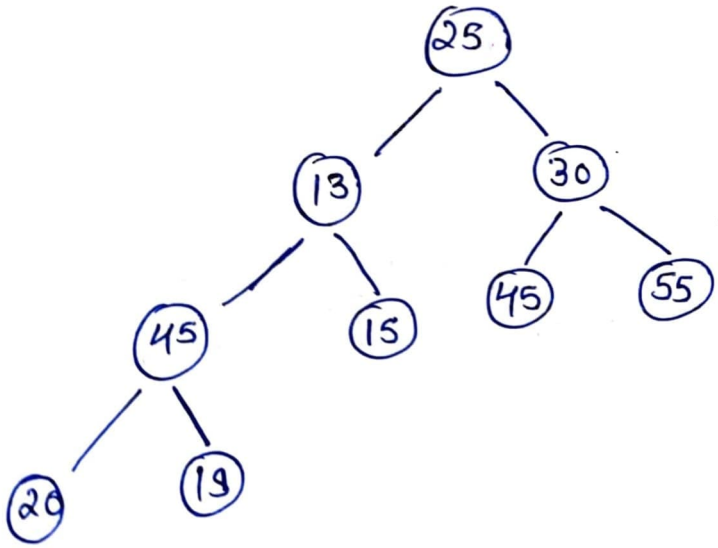
1	2	3	4	5	6	7
15	20	25	30	35	40	45

Max-Heapify (A, i) :→

1. l = left(i) ($\because 2i$)
2. r = right(i) ($\because 2i+1$)
3. if ($l \leq$ heap.size and A[l] > A[i])
4. largest = l
5. else largest = i
6. if ($r \leq$ A.heap.size and A[r] > A[largest])
7. largest = r
8. if largest = i
9. Exchange [A[i], A[largest])
10. Max-Heapify (A, largest)

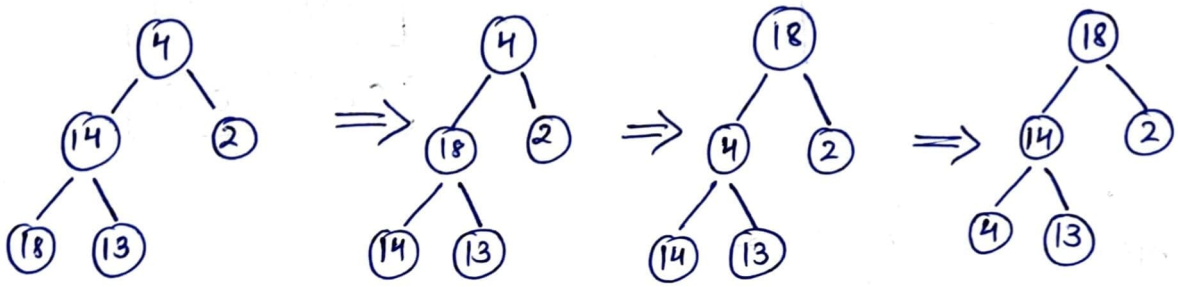
$$T(n) = O(\log_2 n)$$

Q 25, 13, 30, 45, 15, 20, 18, 45, 55



Build Max-Heap(A):→

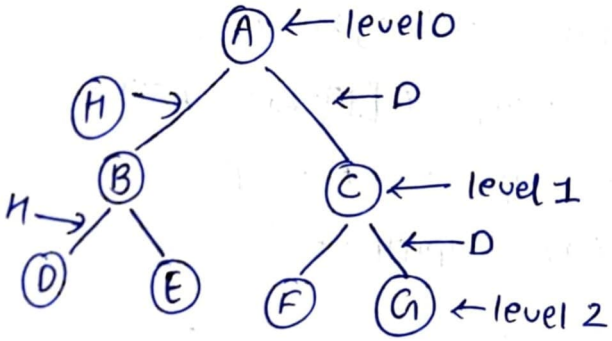
1. A.size = A.length
2. for $i = \left[\frac{A.length}{2}\right]$ to 1
3. Max-Heapify(A, i)



Height ⇒ Bottom to top

Depth ⇒ Top to Bottom

Level ⇒ 3



Note: $H = D = L$

N Node $= 2^{h+1} - 1$

Leaf Node $= 2^h$

Internal Node $= 2^h - 1$

$$\left[\frac{n}{2^{n+1}}\right] = \left[\frac{15}{2^{0+1}}\right] = 7.5 = 8$$

$$\left[\frac{15}{2^{1+1}}\right] = \left[\frac{15}{4}\right] = 3.4 = 4$$

$$\left[\frac{15}{2^{2+1}}\right] = \left[\frac{15}{8}\right] = 1.875 = 2$$

$$\sum_{h=0}^{\log_2 n} \left\lceil \frac{2^n}{2^{h+1}} \right\rceil \times O(h)$$

$$\sum_{h=0}^{\log n} \left\lceil \frac{n}{2^n \times 2} \right\rceil \times n = \frac{n}{2} \sum_{h=0}^{\log n} \frac{h}{2^h}$$

$$\Rightarrow \boxed{\frac{n}{2} \sum_{h=0}^{\infty} \frac{h}{2^h} = 2} = \frac{n}{2} \times 2$$

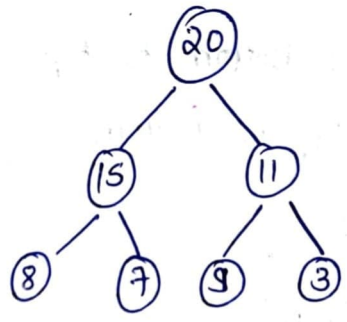
Build Max-Heap

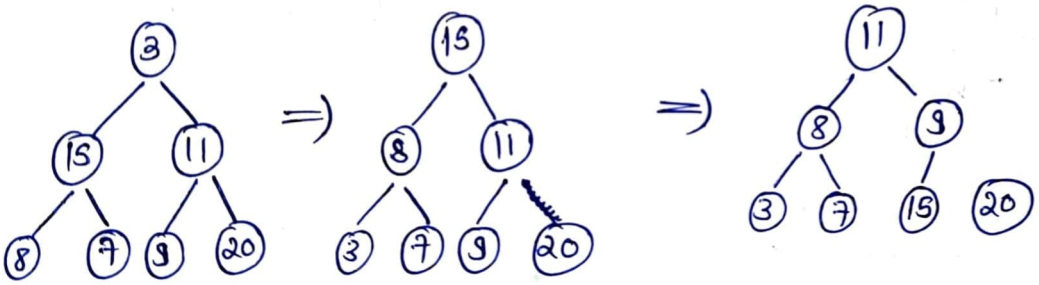
$T(n) \Rightarrow O(n)$

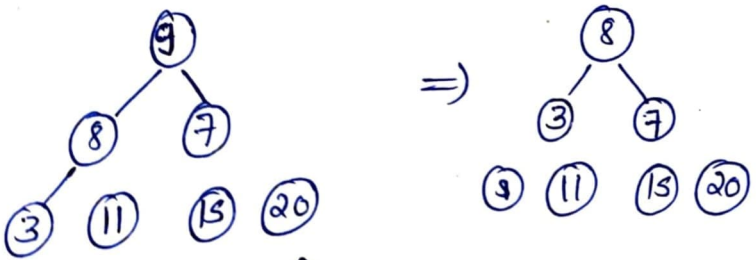
$S(n) \Rightarrow O(1)$

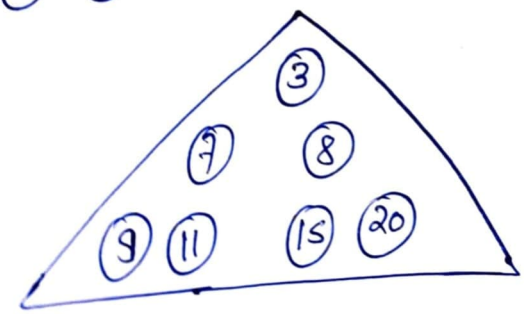
HEAP-SORT (A) :→

1. Build Max-Heap (A)
2. For i = A.Length to 2
3. Exchange A[1] with A[i]
4. A.heapsize = A.heap.size − 1
5. Max Heapify (A, 1)









$$= O(n) + O(n-1) + O(n \log n)$$

$$\boxed{T(n) = O(n \log n)}$$

Heap Sort

Linear Time Sorting Method

→ Counting Sort

→ given input size n

→ take range K(1---K)

2, 1, 2, 3, 1, 3, 2, 4, 5, 4 → highest element +1

→ 1 to K

2	1
3	2
2	3
2	4
1	5
	6

⇒ 1, 1, 2, 2, 2, 3, 3, 4, 4, 5

Time complexity = O(n+k)

Radix Sort

142, 563, 893, 73, 9, 896, 45

1	4	2
5	6	3
8	9	3
0	7	3
0	0	9
8	9	6
0	4	5

⇒

1	4	2
5	6	3
3	9	3
0	7	3
0	4	5
8	9	6
0	0	9

⇒

0	0	9
1	4	2
0	4	5
5	6	3
0	7	3
8	9	3
8	9	6

⇒

0	0	9
0	4	5
0	7	3
1	4	2
5	6	3
8	9	3
8	9	6

unit → pass 1 → pass 2 → pass 3

→ 009, 045, 073, 142, 563, 893, 896

Time complexity = n × k = O(nk)

= O(nk)

Bucket Sort

It works based on floating point number blw 0.0 to 1.0.

Eg:→ 0.73, 0.13, 0.64, 0.39, 0.20, 0.89, 0.53, 0.42, 0.06, 0.94.

Length of Bucket X Key ⟹ 10 x 0.73 = 7.3

Case 1:→ $T(n) = O(n)$

0	0.06
1	0.13
2	0.20
3	0.39
4	0.42
5	0.53
6	0.64
7	0.73
8	0.84
9	0.94

Case 2:→ 0.39, 0.31, 0.34, 0.33

0					
1					
2	0.39	0.3	0.34	0.33	...
3					
4					

$T(n) = O(n^2)$

Shell Sort (modify of Insertion Sort)

0	1	2	3	4	5	6	7	8
23	29	15	19	31	7	9	5	2

23, 7, 15, 19, 31, 29, 7, 5, 2

23, 7, 9, 19, 31, 29, 15, 5, 2

23, 7, 9, 5, 31, 29, 15, 19, 2

23, 7, 9, 5, 2, 29, 15, 19, 3

} Pass 1

2, 7, 9, 5, 23, 29, 15, 19, 31

2, 5, 9, 7, 23, 29, 15, 19, 31

2, 5, 9, 7, 15, 29, 23, 19, 31

2, 5, 9, 7, 15, 19, 23, 29, 31

} pass 2

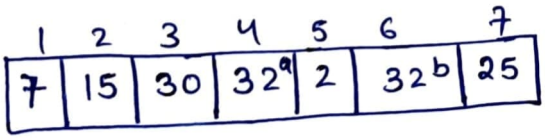
2, 5, 7, 9, 15, 19, 23, 29, 31

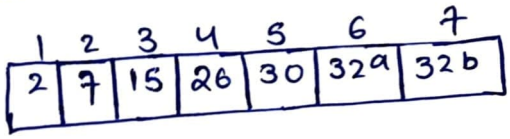
Time Complexity

$O(n^{1.2}$ to $n^2) \Rightarrow O(n^2)$

Stable and Non Stable Algorithm

If the sorting Algorithm after sorting the element does not change the sequence of singular element in which they appear in array it is called stable sorting Algorithm otherwise it is Unstable Sorting Algorithm





Stable Sorting ⇒ Insertion sort, Bucket, Bubble sort, Merge Shell sort, Counting sort, Radix sort

Unstable Sort ⇒ Quick Sort, Selection Sort, Heap Sort, Shell Sort

Inplace and Outplace Sorting Method :→

Sorting technique may require some extra space for comparison and temporary storage for few data elements.

⇒ Inplace → Bubble Sort, Insert, Selection, Quick, heap sort

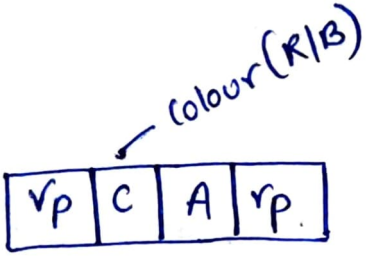
⇒ Outplace → Merge Sort, Bucket, Counting and Radix Sort

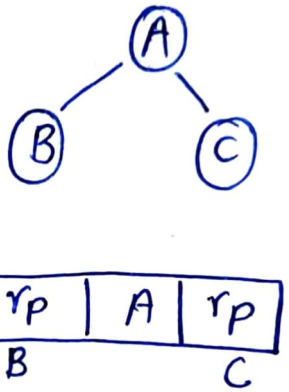
Eg → (stable) $3, 2^A, 2^B, 5$

$2^A, 3, 2^B, 5$ } (Bubble Sort) Pass 1

$\Rightarrow 2^A, 2^B, 3, 5$

Red Black Tree





Self Balancing Binary Search Tree :→

Properties :→

1> Each node is either black or red

2> The root ~~is~~ ~~not~~ always is black.

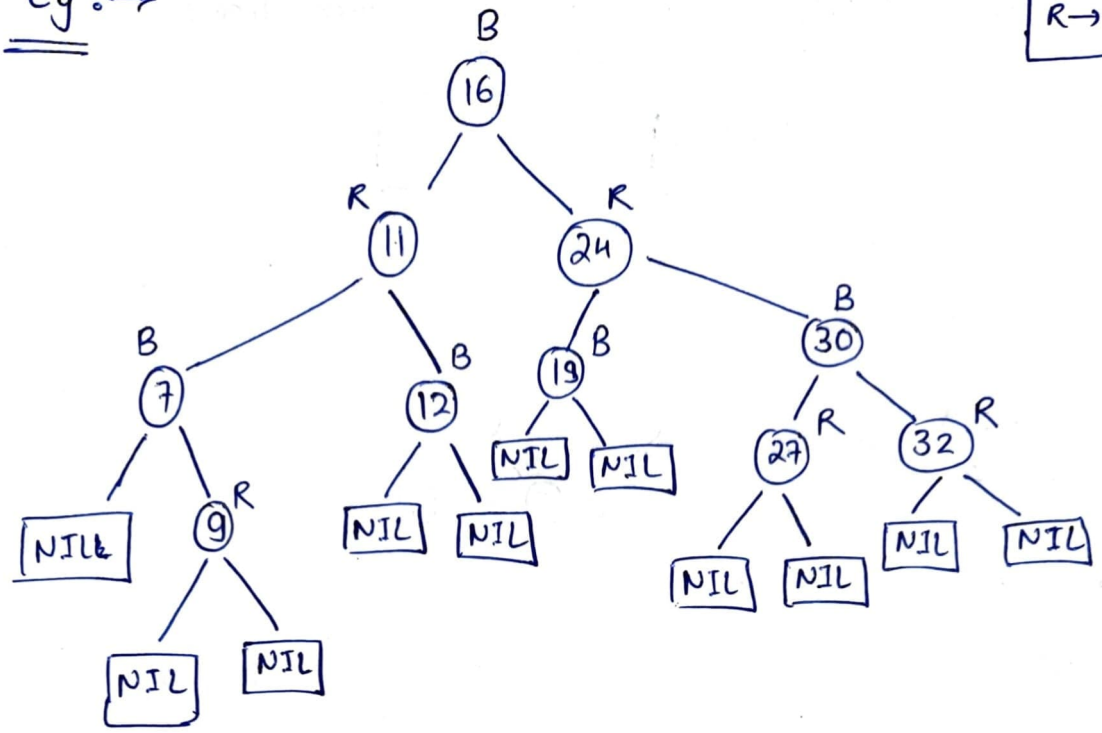
3> Red node cannot have red children (No two red node can be adjacent)

4> Each Path from node to its decendent leaf must have the same number of black nodes

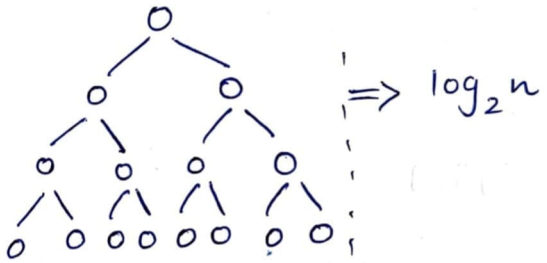
5> All leaf node (null node) are black and do not store any data

Eg :→

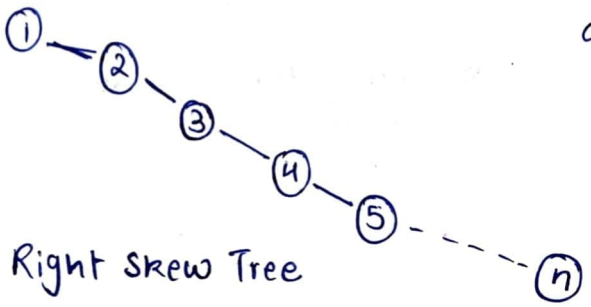
B → Black
R → Red



Binary Search Tree



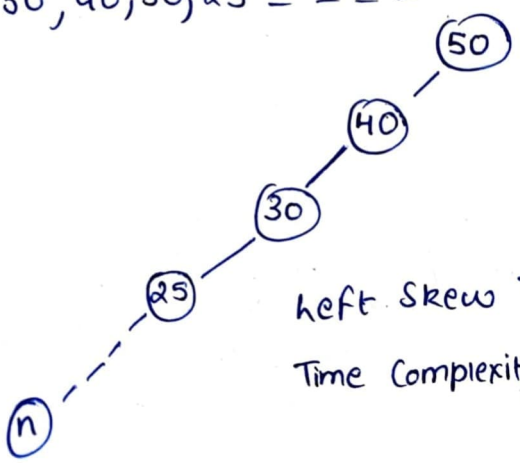
1, 2, 3, 4, 5 ---- n



Right Skew Tree

Time Complexity ⇒ O(n)

50, 40, 30, 25 ---- n



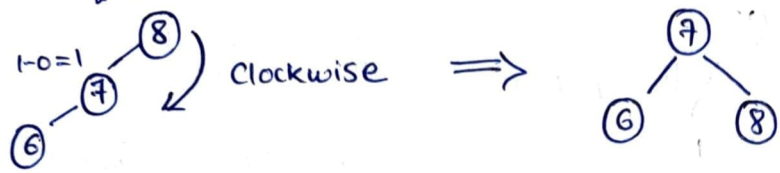
left Skew Tree

Time Complexity ⇒ O(n)

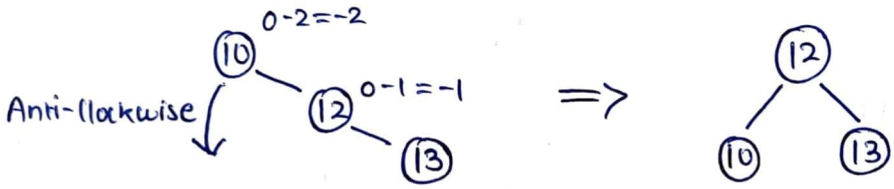
AVL Tree (Balance Binary Search Tree)

Balance factor → $|h_L - h_R| \leq 0, -1, 1$

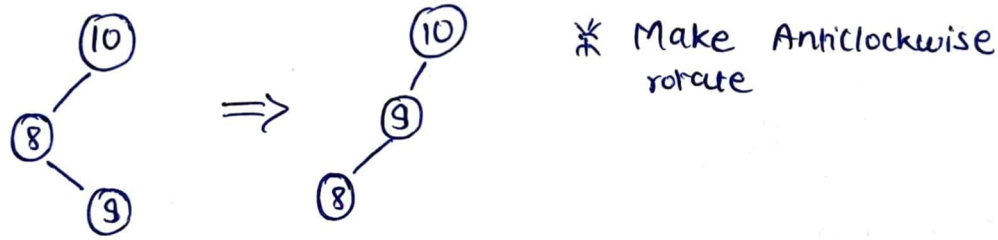
LL :→ 8, 7, 6

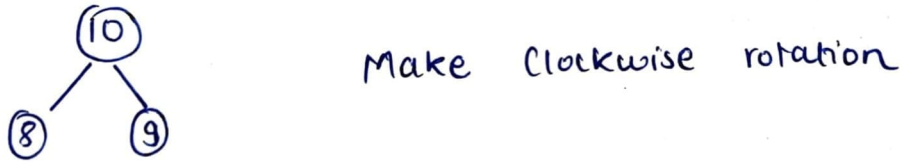


RR :→ 10, 12, 13

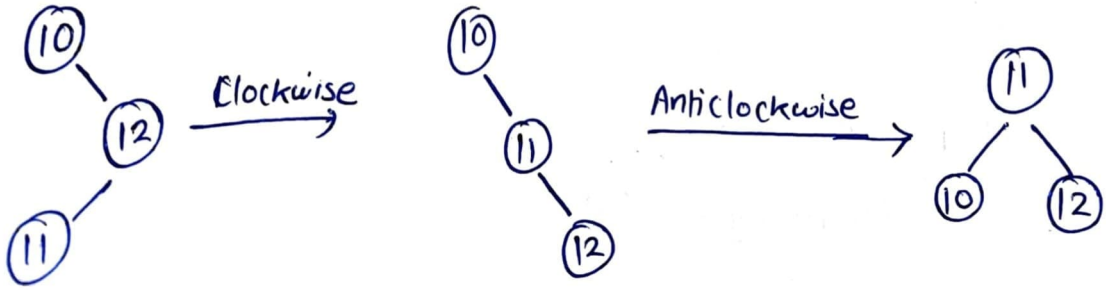


LR :→ 10, 8, 9





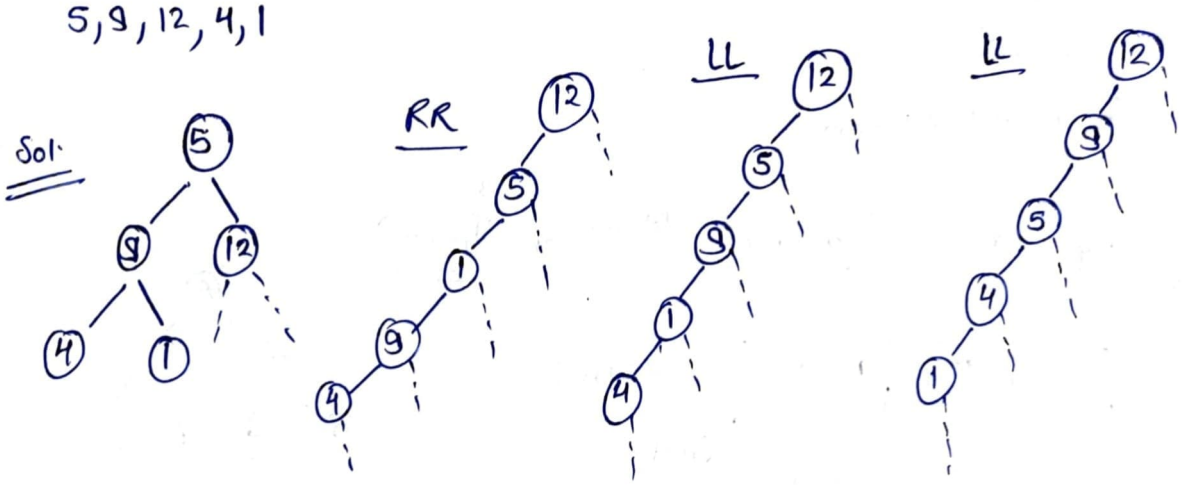
RL :→ 10, 12, 11



Q In a AVL Tree insert the elements :→

5, 9, 12, 4, 1

Sol:



Algo of Red - Black Tree (Insertion) :→

Step 1: Insert the new node a[i] with red color

Step 2: If node is root, then recolour it to black root

Step 3: Case 1: (Red - Red conflicts)

Case 1: if node is red

→ Recolor the parent and unde to black and Granparent to red.

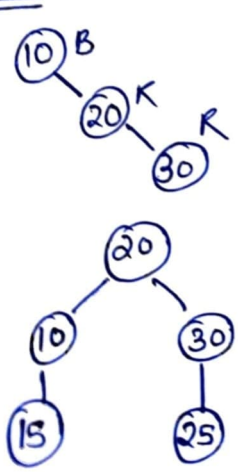
→ Then, repeat the fix-up process from grandparent

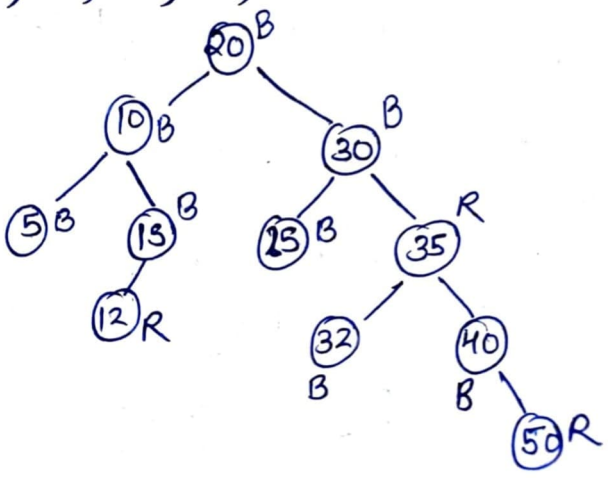
Case 2: (uncle is black or null)

→ Perform rotation (LL, LR, RL, RR)

→ Recolor the node accordingly.

Eg→ 10, 20, 30, 15, 25, 5, 12, 35, 40, 32, 50



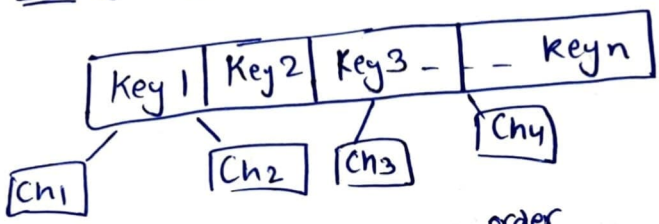


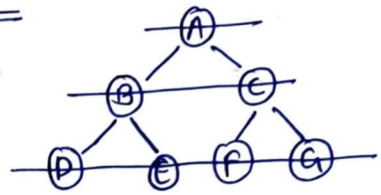
$\lceil \log_{11} \rceil = \lceil 34 \rceil = 4$

$= O(\log n)$

Insertion / deletion

Binary Tree (Dynamic Multilevel Index)





→This is the balanced m-way order tree. It is based on BST (Binary Search Tree)

→ It mantains sorted data.

→ All leaf must be at the same level.

Ex:→ m=5 - order

Internal node [max key (m-1) = (5-1) = 4]

[min key $\left\lceil \frac{5}{2} \right\rceil - 1 = \lceil 2.5 \rceil - 1 = 3 - 1 = 2$]

min. key of root node = 1

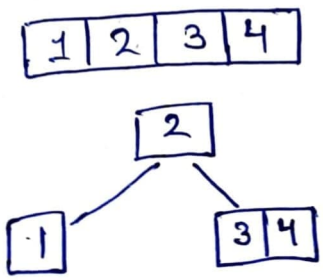
Q Insertion the following key into B. Tree if order of B. Tree is 4.

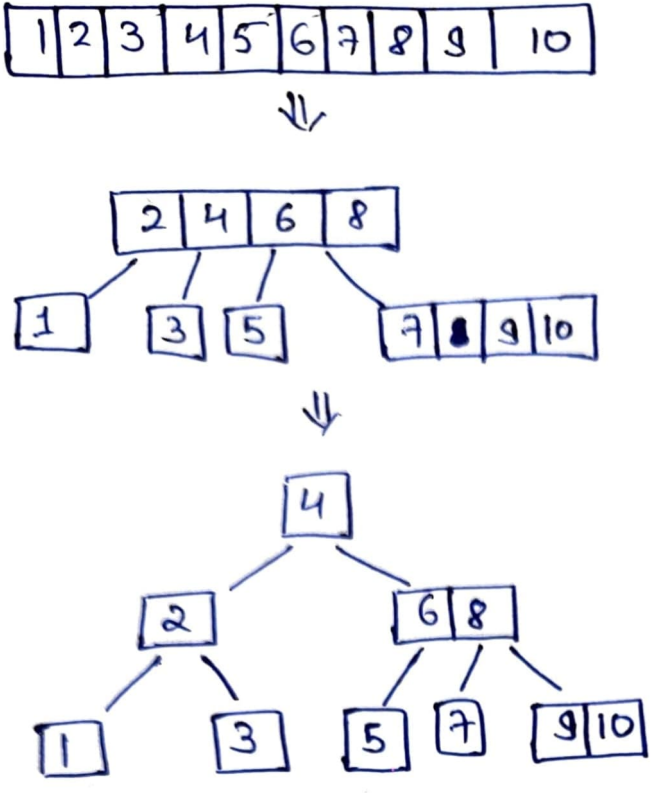
Sol. m=4, max keys = [m-1) = (4-1) = 3

min keys. = $\left\lceil \frac{4}{2} \right\rceil - 1 = 2 - 1 = 1$

~~node~~ root node 1 always

B. Tree





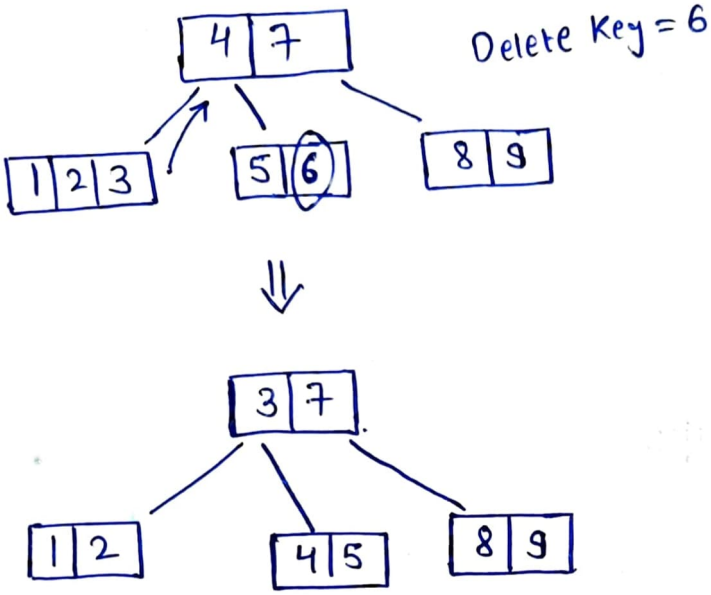
Q Insert the keys = 78, 52, 81, 40, 33, 90, 85, 20, 38 in B. Tree, order $m = 3$

Delete of B-Tree

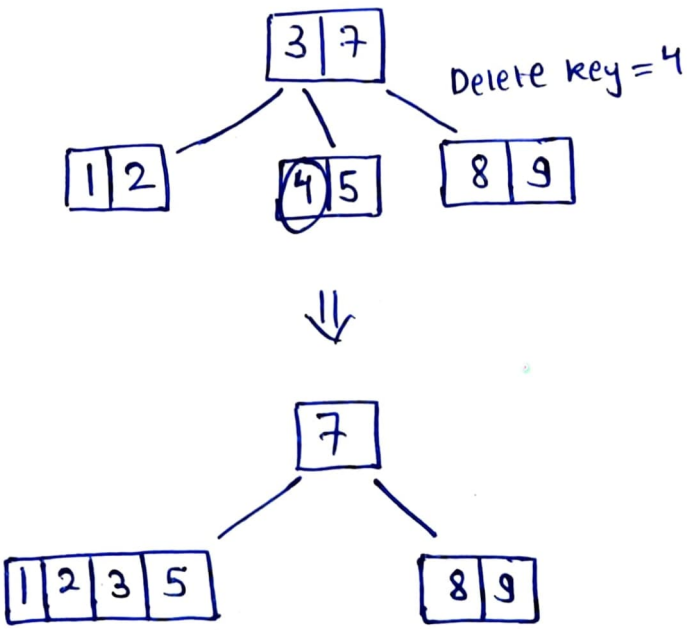
$p = 5$, max keys $= (5-1) = 4$

min key $= \left[\frac{5}{2}\right] - 1 = 2$

Case 1 :→ (Borrow Method)

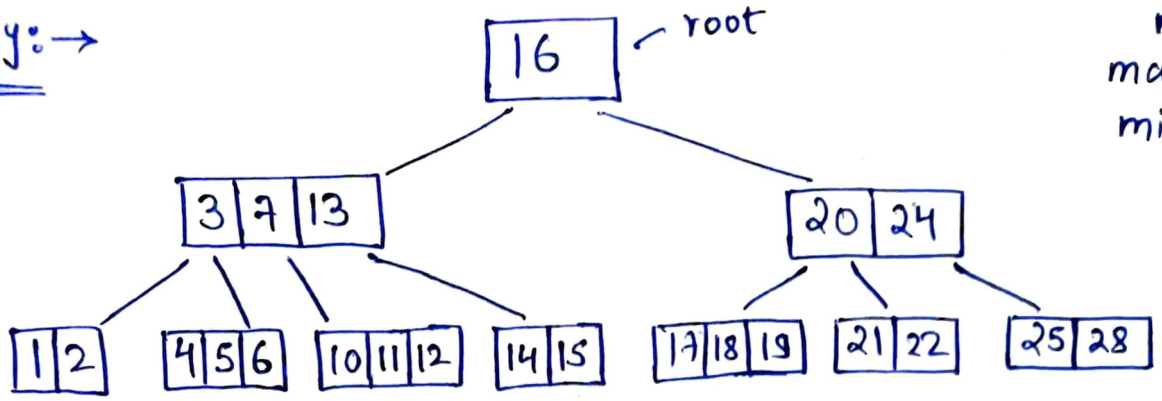


Case 2 :→ Coalesce method (~~Borrow~~ Merge)



Time Complexity = $O(\log n)$ → Insertion, Deletion

Eg:→



$m = 6$

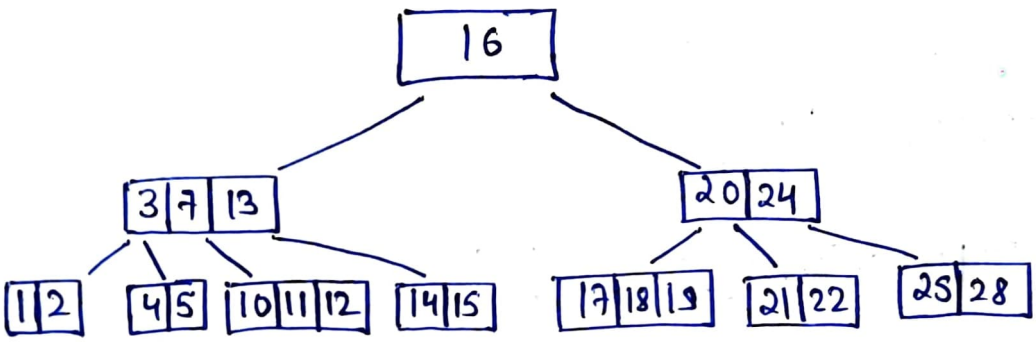
$max = 6-1 = 5$

$min = \left\lceil \frac{6}{2} \right\rceil - 1$

$= 3-1 = 2$

Delete Key = 6, 13, 7, 4, 2

1) Delete Key = 6

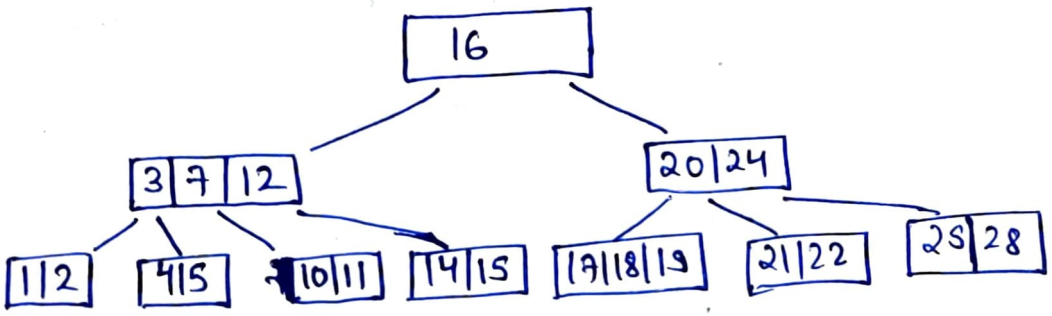


✱ 10, 11, 12, (13), 15, 16

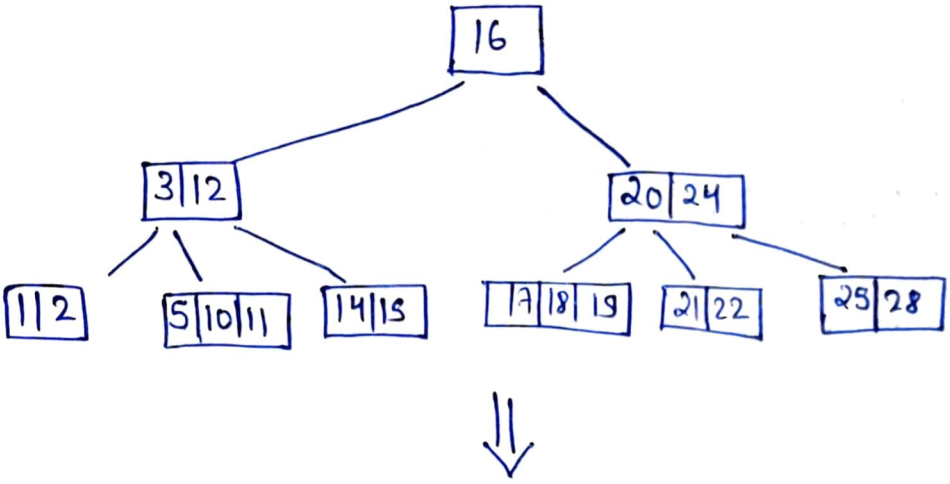
Inorder predessor

Inorder Sucessor

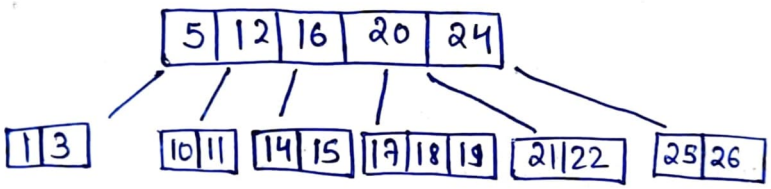
2) Delete Key = 13



3) Delete Key = 7, 4



4) Reduce the Search Time , Delete Key = 2

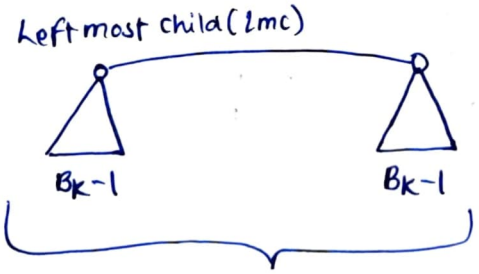


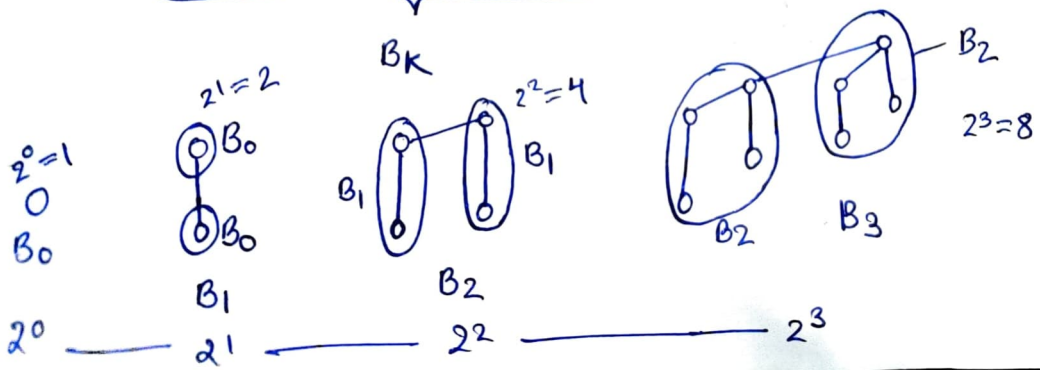
Bionomial Heap

Bionomial Heap is a collection of Bionomial Tree. The Bionomial Tree B_k is an odered tree define recursive.

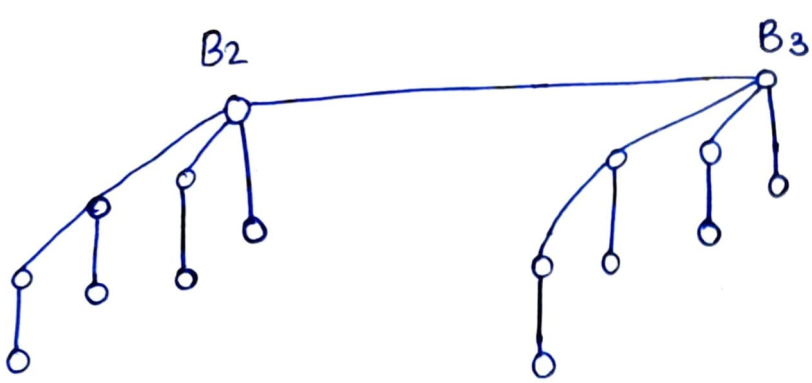
The Bionomial tree B_0 consist of single node

The Bionomial tree B_k consist of 2 Bionomial tree B_{k-1} that linked together.





$B_4 \longrightarrow B_3 + B_2$



B_k

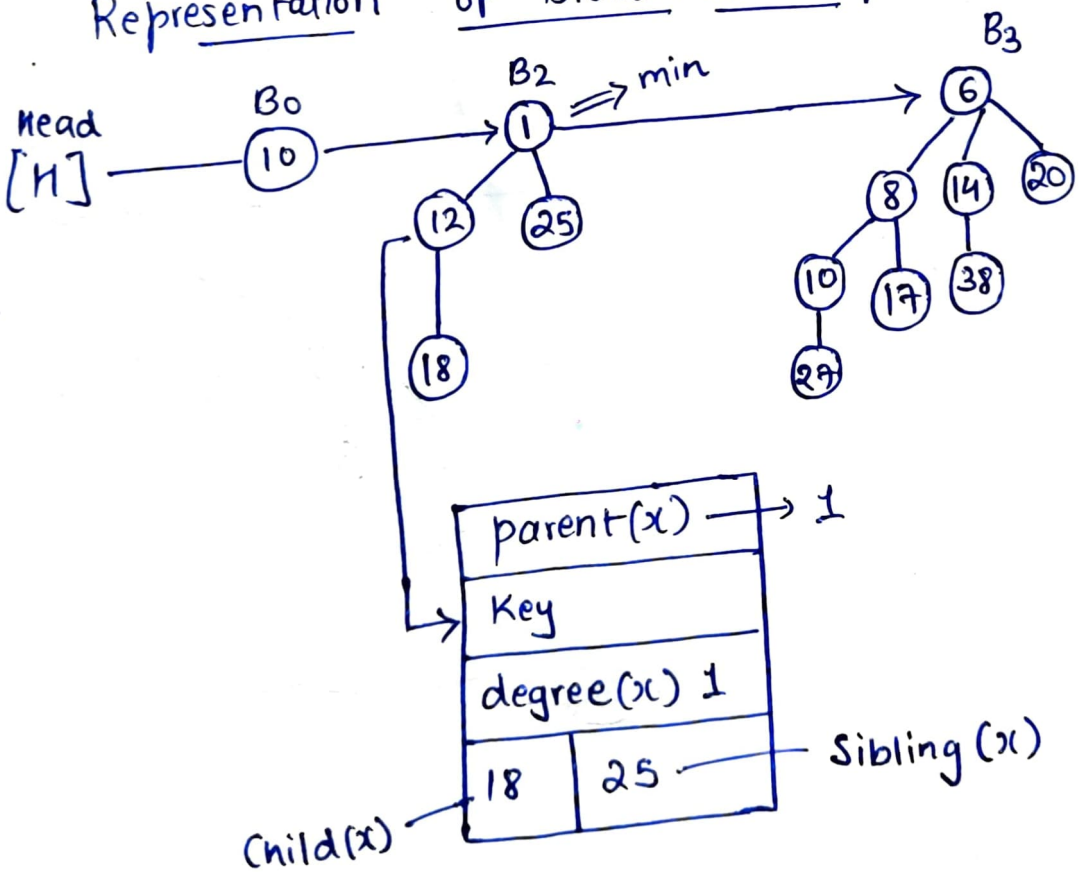
$2^k = 2^4$

$= 16$

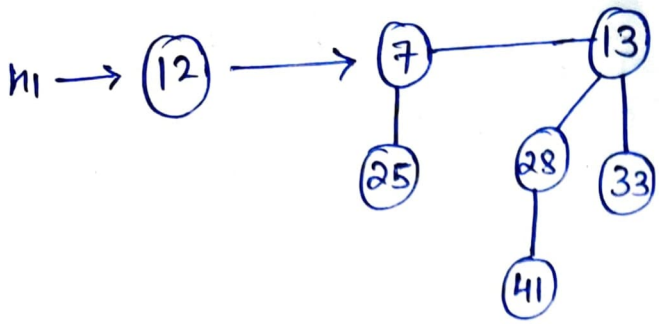
A Binomial Heap H is a set of binomial tree that satisfy the following Binomial Heap properties :⟶

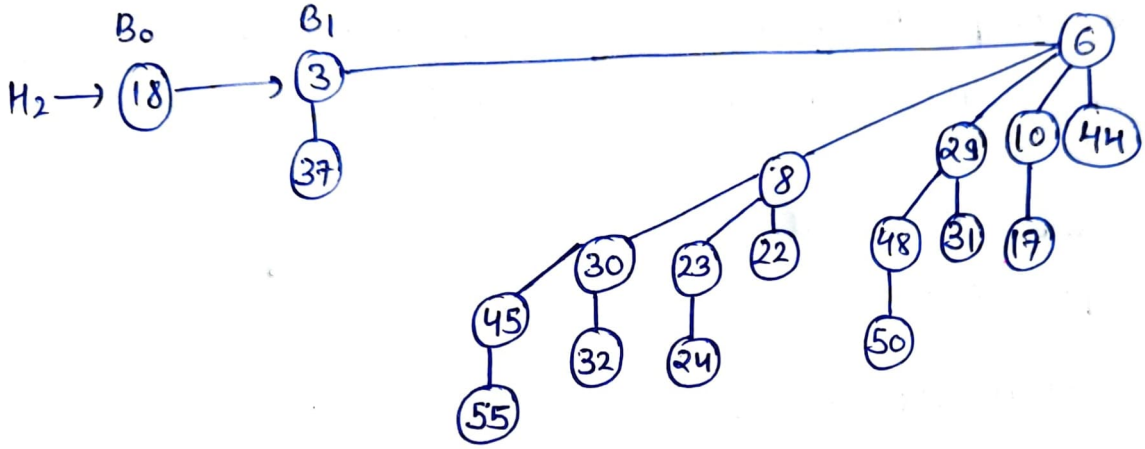
1) Each ~~node~~ Binomial tree H obeys that Min-Max Heap properties.

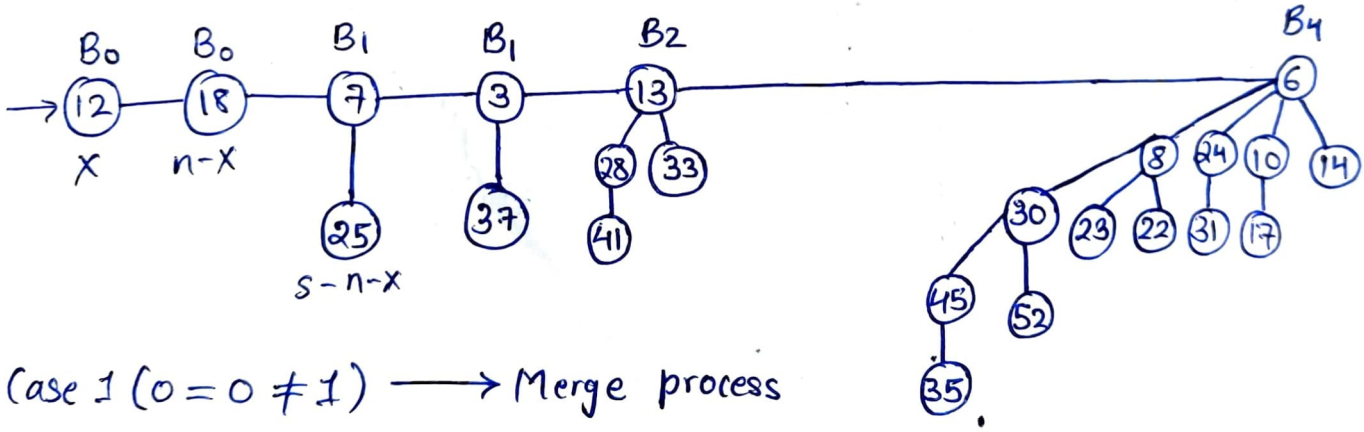
Representation of Binomial Heap



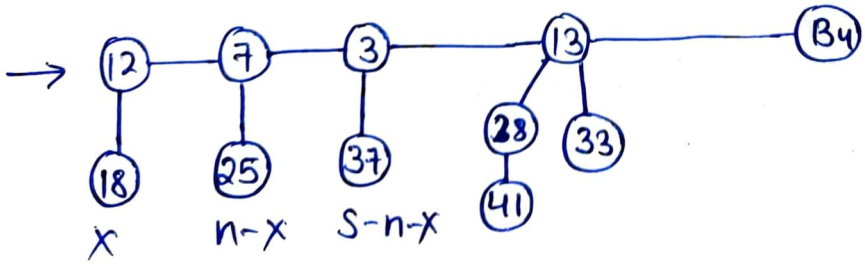
Union of two Binomial Heap



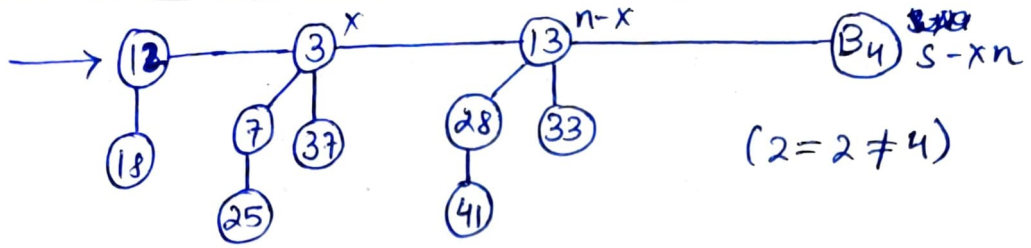




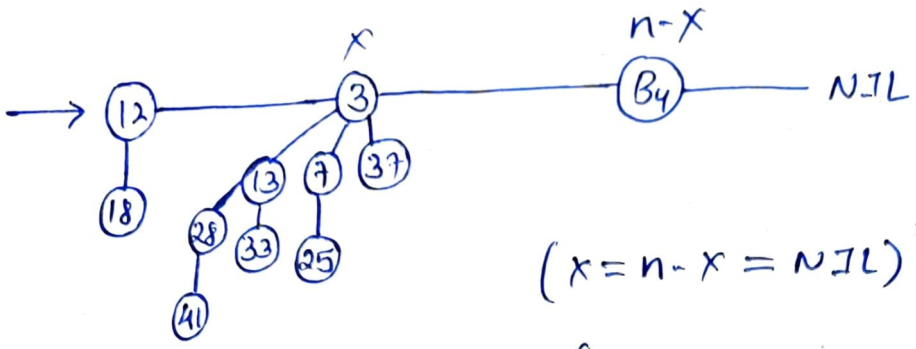
Case I (0=0≠1) ⟶ Merge process



Case II (1=1=1) → pointer shift are position in n+1 side



(2=2≠4)



(x=n-x=NIL) Stop

(2≠4≠NIL)

Algo of Bionomial min Heap Union operation

1) Merge the root list of Bionomial Min Heap (H_1, H_2) into a single linked list that is stored in non decreasing order of their degree.

H_1: $B_0 \longrightarrow B_1 \longrightarrow B_2$ (root list)

H_2: $B_0 \longrightarrow B_2 \longrightarrow B_4$ (root list2)

$\rightarrow$ $B_0 \longrightarrow B_0 \longrightarrow B_1 \longrightarrow B_2 \longrightarrow B_2 \longrightarrow B_4$

x n-x s-n-x

2) Link the root of eaual degree until almost one root remain of each degree (de different)

If (degree(x) ≠ degree(next-x) or degree(x) = degree(n-x) = [sibling[next-x]] → move one pointerin right position

If (degree(x) = degree(n-x) ≠ degree(Sibling(n-x)) → Apply the merge process.

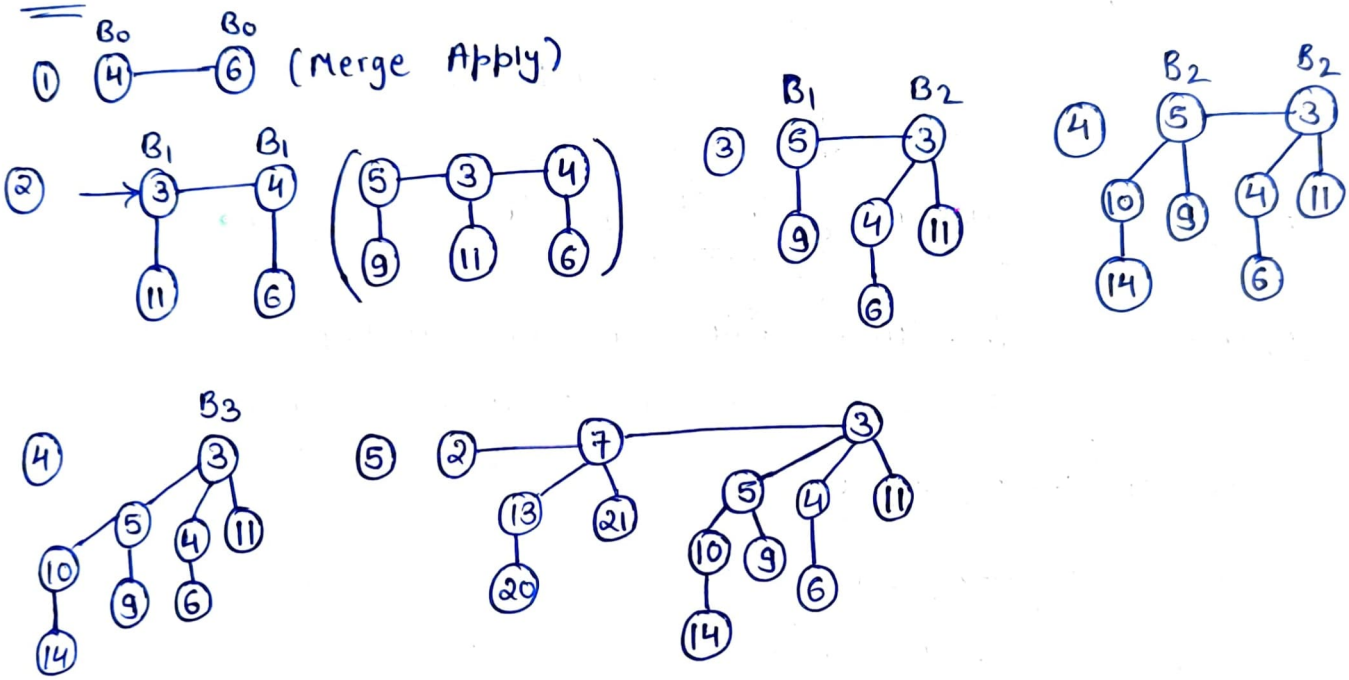
Algo of Bionomial Min Heap Union Operation

→ Insertion a node into Bionomial Heap (min Heap)

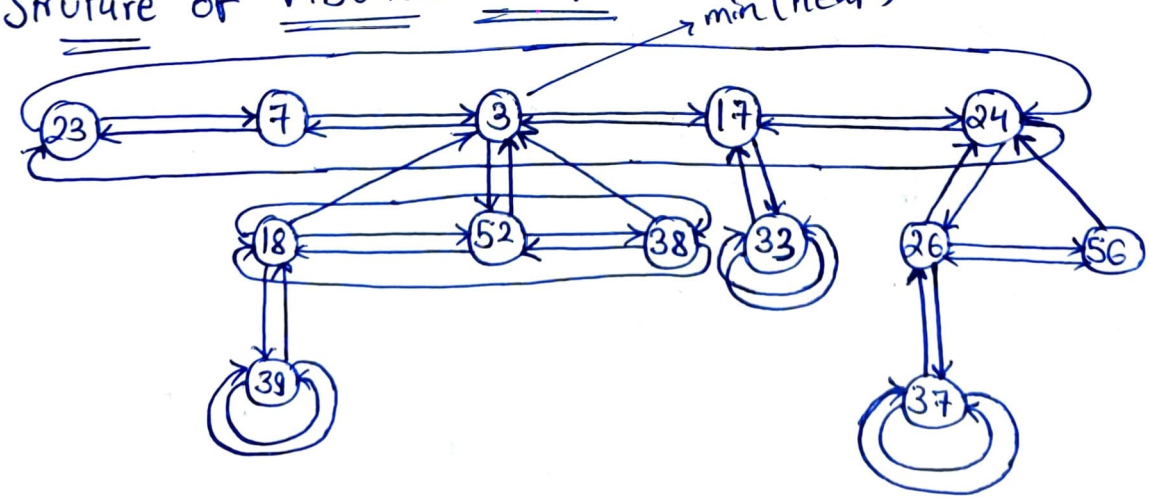
Bionomial Heap H' → Containing new element

→ Apply union of two Bionomial Heap (H ∪ H')

Eg:→ 4, 6, 3, 11, 9, 5, 14, 10, 21, 7, 13, 20, 2



Struture of Fibonoci Heap



p(x), degree(x) — 1

child(x), left(x), right(x)

Operation of Fibonacci Heap

- Create / Make Fibonacci Heap
- Insert (H, x)
- Find min
- Union

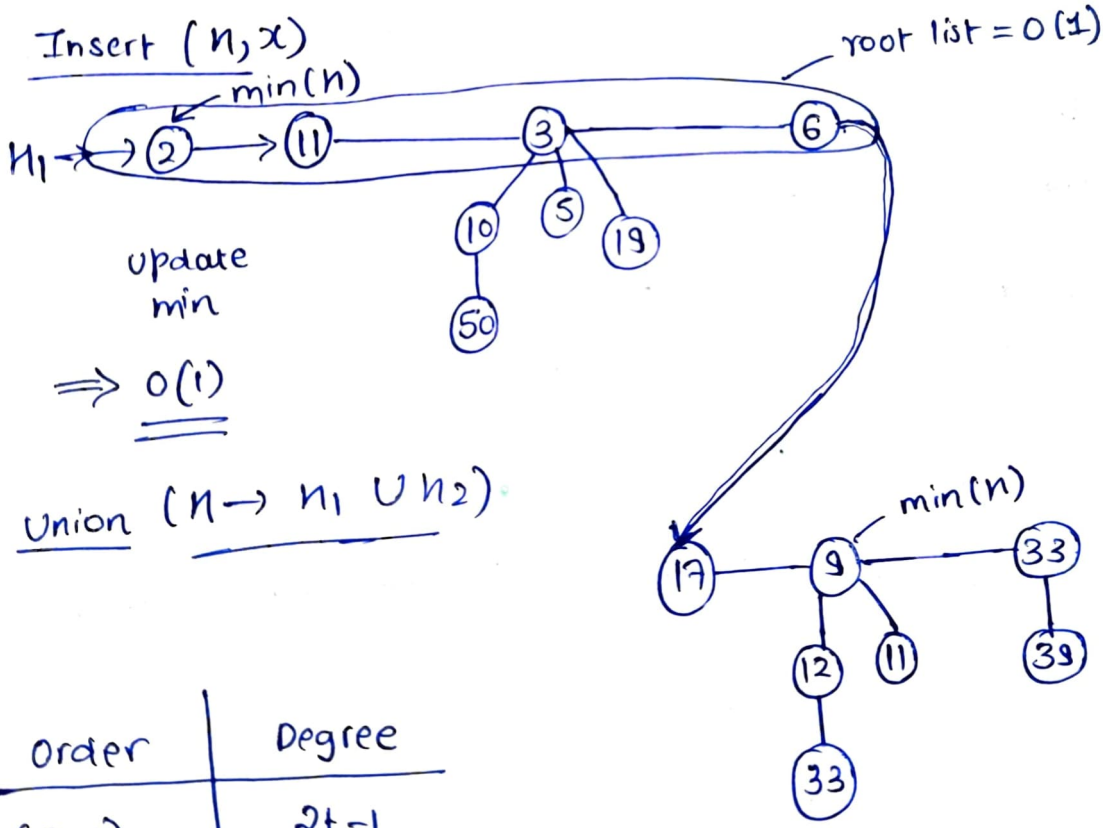
Create Heap ()

heap (H) = NIL

⇓

O(1)

Insert (H, x)



update min

⇒ O(1)

Union (H → H1 ∪ H2)

Order	Degree
(P-1)	2t-1
$\left[\frac{P}{2}\right]-1$	t-1

2x degree = Order

Amortized Analysis

→ In Amortized Ananlysis time required to perform a seaurence of operation is averaged overo all the operations performed.

→ It is generally used in hash table stack (multi pop and disjoint set) operation.

- → Aggregate method
- → Accounty method
- → Potential method

Eg:→ 201 items

1 item → 1 cost

1 item → 200 cost

Asymptotic Analysis	Amortized Analysis
201 X 200	1 x 200 + 200
= 40000	= 400

Q Create a Bionomial heap with 15 nodes.

Bucket Sort

It works based on floating point number blw 0.0 to 1.0.

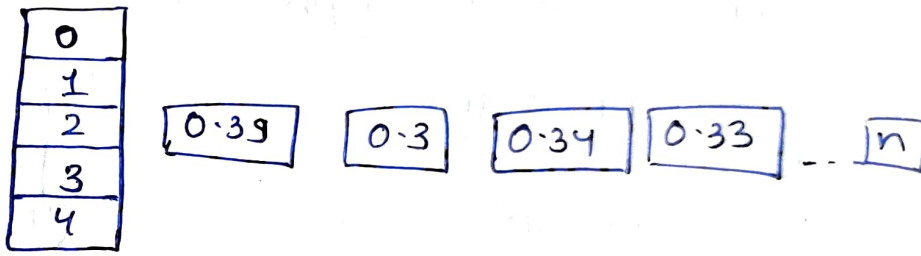
Eg:→ 0.73, 0.13, 0.64, 0.39, 0.20, 0.89, 0.53, 0.42, 0.06, 0.94.

Length of Bucket X Key ⟹ 10 x 0.73 = 7.3

0	0.06
1	0.13
2	0.20
3	0.39
4	0.42
5	0.53
6	0.64
7	0.73
8	0.84
9	0.94

Case 1:→ $T(n) = O(n)$

Case 2:→ 0.39, 0.31, 0.34, 0.33

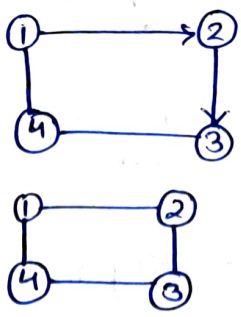


$T(n) = O(n^2)$

Graph Theory (Algorithm) → G = (V, E)

→ Direct Graph

→ Indirect Graph



→ It is non-linear data structure

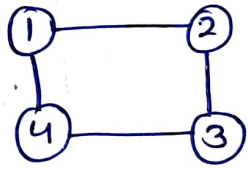
→ V → Set of Vertex V = (1, 2, 3, 4)

→ E → set of Edges E = {1, —2}
{2-3}
{3-4} {4-1}

Representation of Graph :→

→ Add Matrix

→ Add List



add

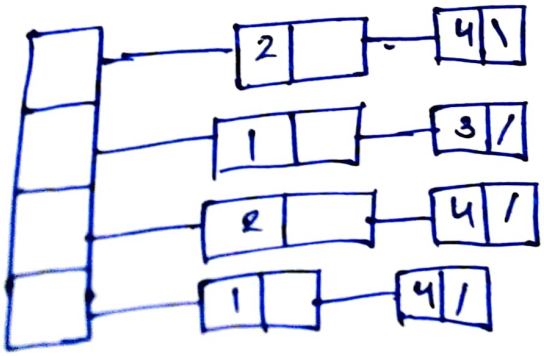
Adj matrix :→

	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	0	1	0	1
4	1	0	1	0

n×n

⇒ O(n²) ⇒ O(v²)

Adj List :→

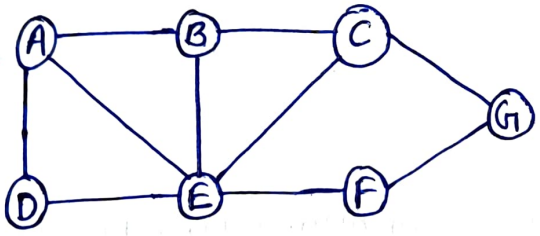


→ If Graph is dense then usage Adj. Matrix otherwise it will be use Adj. List.

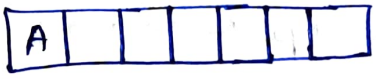
GRAPH SEARCH METHOD

- BFS (Breadth First Search)
- DFS

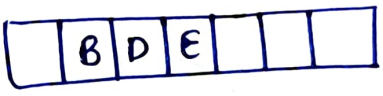
BFS (Breadth First Search)



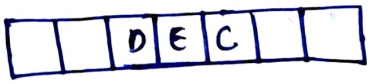
Queue



→ put all elements

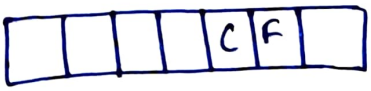


A (Dequeue)

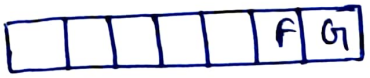


A, B

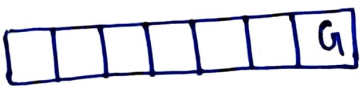
A, B, D



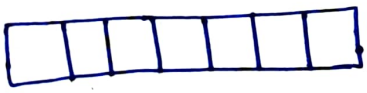
A, B, D, E



A, B, D, E, C



A, B, D, E, C, F

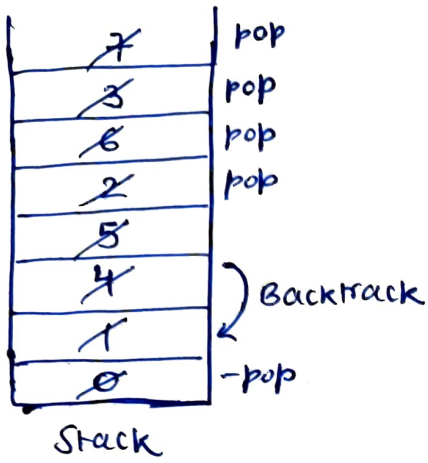


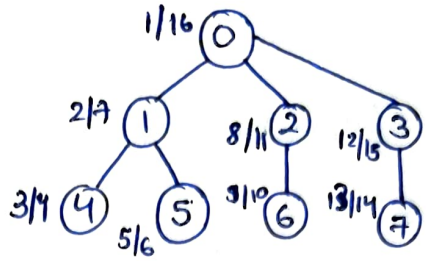
A, B, D, E, C, F, G

$T(n) = O(V+E)$

→ BFS is based on level order Method.

DFS (Depth First Search)





Visited nodes = 0, 1, 4, 5

GREEDY METHOD

The greedy method of algorithm is a problem solving process to use optimisation of a large problem.

→ It makes ~~globally~~ locally optimal choice of each step with the hope of finding a global optimal choice.

Characteristics of Greedy Method

1) Optimization problem
2) Sequencial Decision Making
3) Greedy Choice Property
4) Optimal Substructure
5) Feasible ~~Op~~ Solution

→ Greedy method ~~is~~ does not give ~~guaranteed~~ exact solution. at every time.

Greedy Methods

- Minimum Spanning Tree (Kruskal and prism Algo)
- Single source Shortest path (DiJsktra and Bellman ford Algo)
- Fractional Knapsack problem
- Activity selection problem

#

A ~~connected~~ corrected Subgraph ~~edge~~ H of a given graph said to be spanning tree if :→

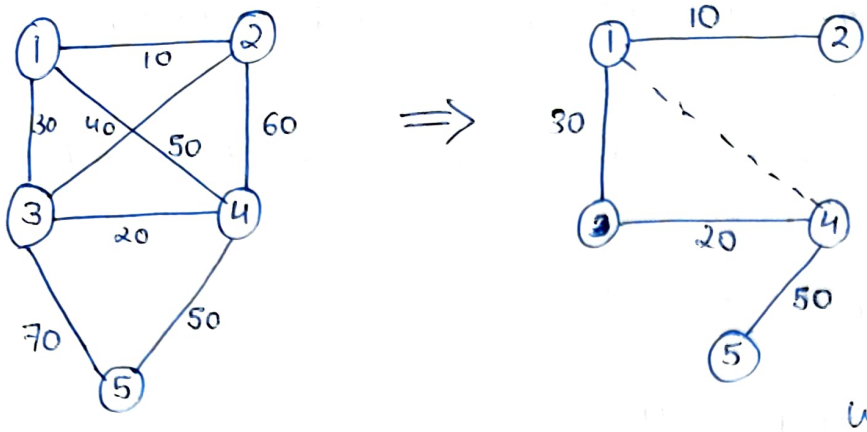
1) ~~If~~ H should be contain or vertex of graph

2) H should contain (1-H)

3) H should be corrected.

KRUSKAL ALGORITHM (Edge Based)

$E = V - 1$

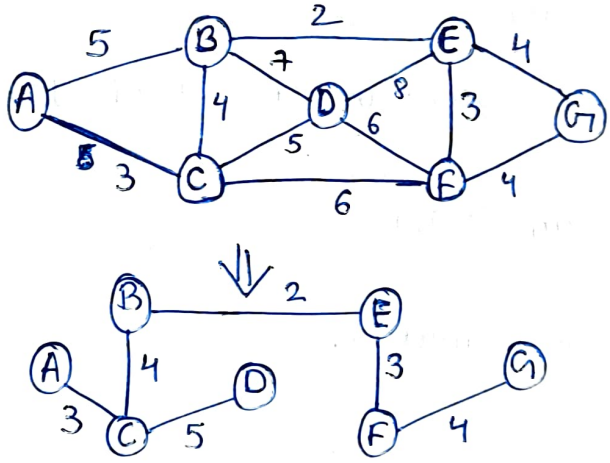


weight = 10 + 20 + 30 + 50

= 110

$T(n) = O(E \log V)$

Q Calculate the minimum weight in a Kruskal algorithm in given graph :→



weight = 3 + 4 + 5 + 2 + 3 + 4

= 21

Advantage of Kruskal Algorithm

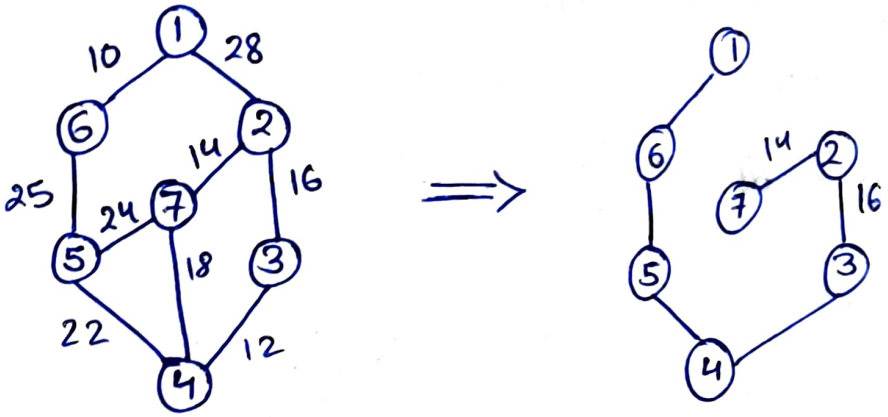
→ It is easy to implement

→ It is faster than prism algorithm

DisAdvantage of Kruskal Algorithm

→ Sometime graph may be disconnected

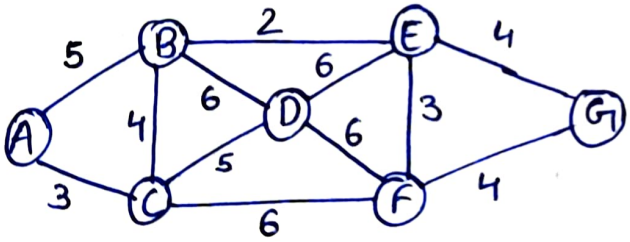
Prism Algorithm



Weight = 10 + 25 + 22 + 12 + 16 + 14

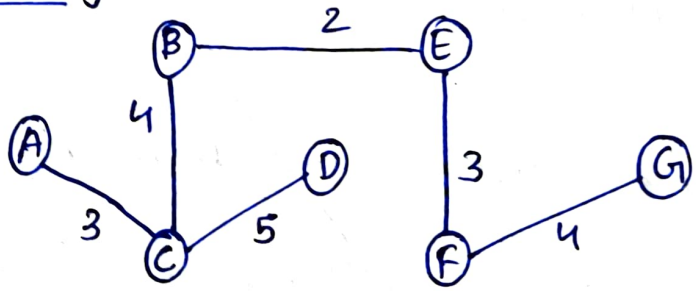
= 99

Q Calculate the main Spanning tree through Kruskal an a Prism Algorithm



Kruskal Algo:

=)



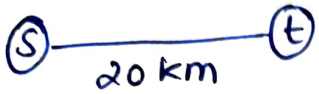
weight = 19

Prism Algo:

Single-Source Shortest Path

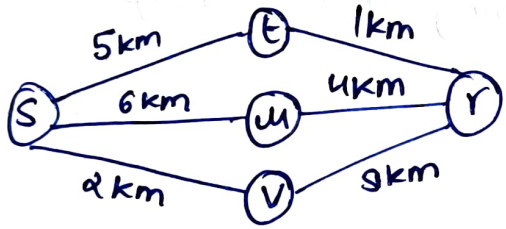
A Shortest Path from Vertex edge to be is defined as any path p with weight

Weight, $w(P) = \delta(s,t)$



Single-Source Shortest Path Problem :→

Given graph $G = (V, E)$ will find out a shortest path from given source vertex $s \in V$ to every vertex, $\forall$ vertex $\in V$



∴ Always Choose Shortest Path

Methods of Greedy Algorithm :→

→ Minimum Spanning tree (Kruskal and Prism Algo)

→ Single Source Shortest Path (Dijkstra and Bell)

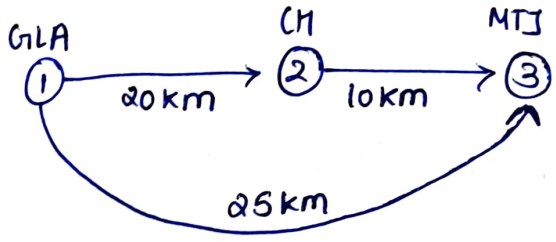
→ Fractional Knapsack problem

→ Activity Selection Problem

Relax function (u, v, w) :→

if d(u) + w(u,v) < d(v)

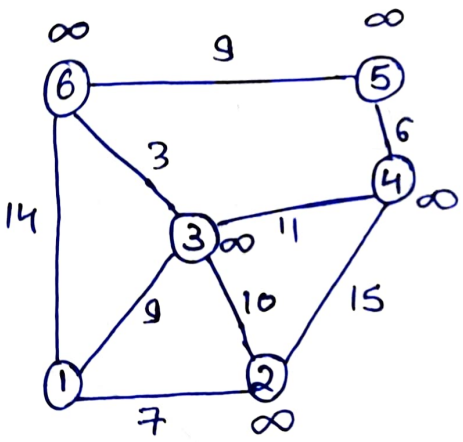
d(v) = d(u) + w(u, v)



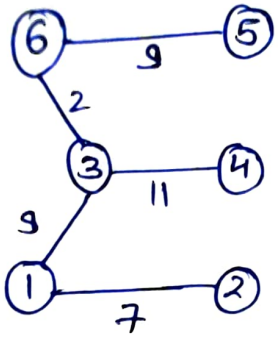
20+10 < ∞

0+25 < 30 ∴ True

DIJKSTRA ALGORITHM

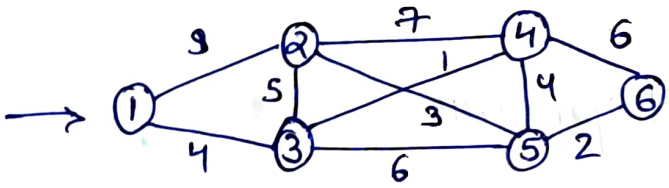


Source Code	Destination Code				
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
1	7	9	∞	∞	14
1, 2	(7)	9	22	∞	14
1, 2, 3	(7)	(9)	20	∞	11
1, 2, 3, 6	(7)	(9)	20	20	(11)
1, 2, 3, 6, 4	(7)	(9)	(20)	20	(11)
1, 2, 3, 6, 4, 5	(7)	(9)	(20)	(20)	(11)

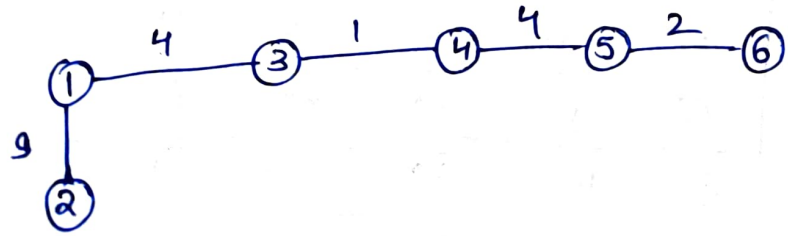


Source Code	Destination Code				
1	2	3	4	5	6
0	7	9	20	20	11

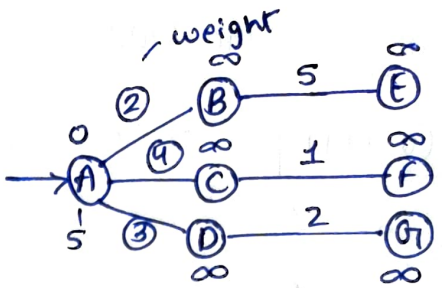
Q Calculate the single shortest path (SSP) using Dijkstra Algorithm



Source Code	Destination Code				
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
1	9	4	∞	∞	∞
1, 3	9	(4)	5	10	∞
1, 3, 4	9	(4)	(5)	9	13
1, 3, 4, 5	9	(4)	(5)	(9)	11
1, 3, 4, 5, 2	(9)	(4)	(5)	(9)	11
1, 3, 4, 5, 2, 6	(9)	(4)	(5)	(9)	(11)



Dijkstra Algo (G, w, S)



→ Create Vertex set Q

→ for each Vertex V in graph

→ dist[V] = ∞
→ add V to Q
→ dist[source] = 0

} Buid Min Heap } O(V)

→ while Q is not empty —— O(V)

→ u = extract_min[Q] —— O(log V)

→ for each neither V of u

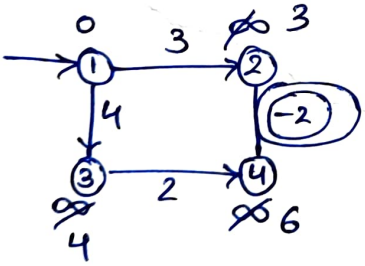
→ Relax (u, v, w)

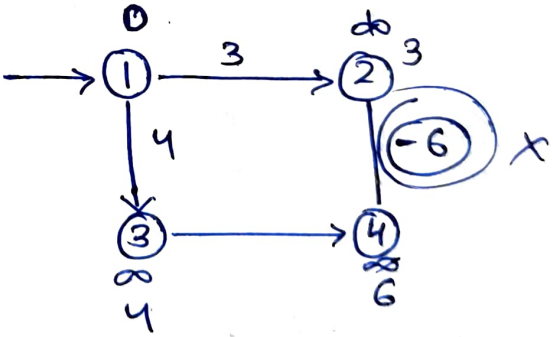
total time complexity

$T(n) = O(E \log V)$

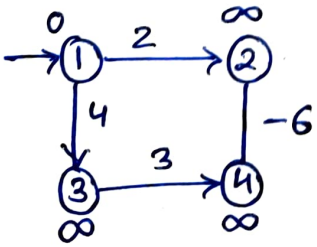
Disadvantage of ~~Dis~~ Dijkstra Algorithm

→ It does not work in negative edge value at everytime.



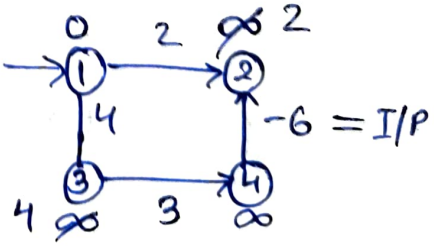


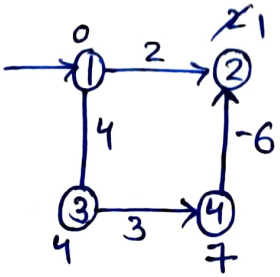
Belman Ford Algorithm

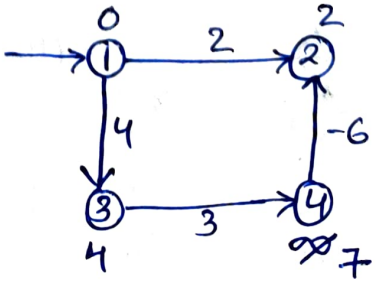


(V-1) Relax

(4-1) = 3 K

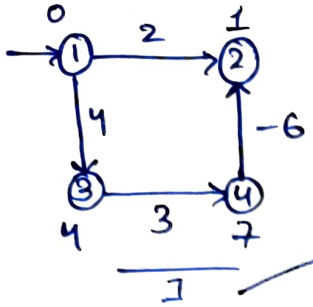






Negative weight cycle

$\boxed{(v-1)+1}$ → To check negative weight cycle



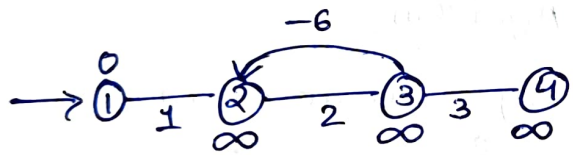
→ No cycle Exist

Comparison between Dijkstra and Bellman

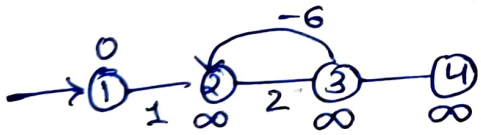
	Dijkstra	Bellman
+Edge	✓	✓
-Edge	✓/X	✓
-WC	X	X

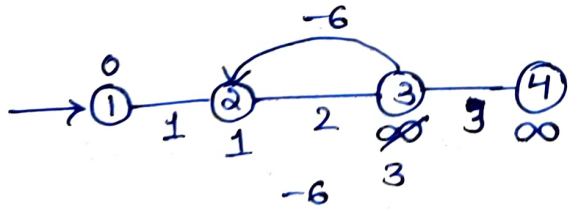
– to identify (– wt) or not

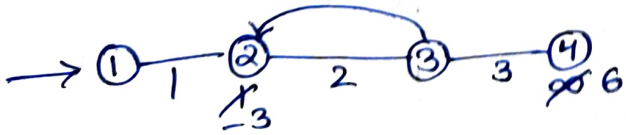
Q

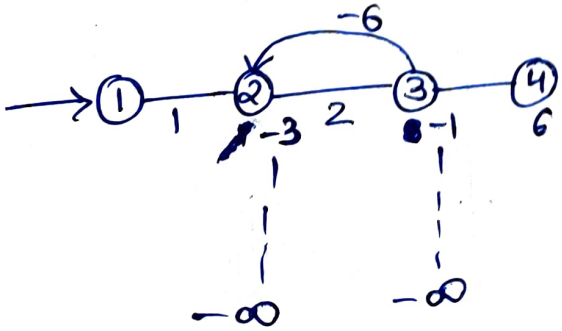


(v-1) Relax









Bellman ford Algo (G, w, s) :→

Initialize Single Source (G, s)

① for i=1 to |G.V|−1 — O(V)

② for each Edge (u,v) (G.V) ∈ G.E — O(E)

③ Relax (u, v, w)

④ for each Edge (u, v) ∈ G.E

⑤ if (d(v) > d(u) + w (u.v)
return false, — O(E)

⑥ Otherwise
return true

Total Time = O(VE) + O(E)

$$T(n) = O(VE)$$

Activity selection Problem :→

Each activity a_i has starting time = S_i
and finishing time f_i $\quad 0 \leq s_i \leq f_i \leq \infty$

- It is Bellman Optimization problem that involves selecting a max no. of non-conflicts activity from given set.

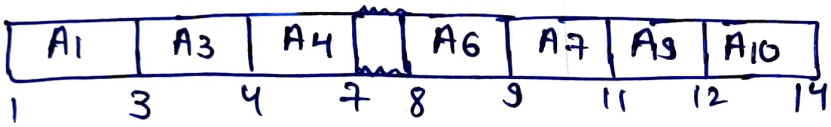
Q *

Q

i	A_1	A_2	A_3	A_4	A_5	A_6	A_7	A_8	A_9	A_{10}
Si	1	2	3	4	7	8	9	9	11	12
Fi	3	5	4	7	10	9	11	13	12	14

Step 1:→ Sort the activities in Ascending Order of their Fi

i	A_1	A_3	A_2	A_4	A_6	A_5	A_7	A_9	A_8	A_{10}
Si	1	3	2	4	8	7	9	11	9	12
Fi	3	4	5	7	9	10	11	12	13	14

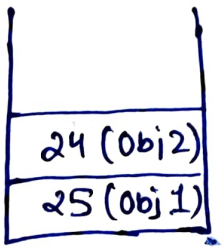


Fractional Knapsack Problem

m=20

Object	1	2	3
Profit	25	24	15
weight	18	15	10

∴ Method 1 (Profit Based)



$20 - 18 = 2$

$1 \times 25 + \frac{2}{15} \times 24 + 0 \times 15$

$25 + 3.2 \Rightarrow$ **28.2**

$$x_i = \frac{\text{Remaining Back-Capacity}}{\text{Total weight}}$$

∴ Method 2 (Min weight)

Obj	1	2	3
P	25	24	15
w	18	15	10

Obj 2 $(\frac{10}{15})$
Obj 3 (10)

$= 1 \times 15 + \frac{10}{5} \times 24 + 0 \times 25$

$\Rightarrow$ 31

∴ Method 3

Obj	1	2	3
P	15	24	15
w	18	15	10
P/w	1.4	1.6	1.5

Obj 3 $(\frac{5}{10})$
Obj 2 (15)

$= 24 \times 1 + \frac{5}{10} \times 15 + 1.4 \times 0$

$\Rightarrow$ 31.5

GREEDY KNAPSACK ALGORITHM :→

① for i=1 to n
② Compute $\frac{P}{wi}$ } O(n)

③ Sort the objects in the non-increasing order if $\frac{Pi}{wi}$ } O(log n)

④ for i = 1 to n
⑤ if (M > 0 and wi ≤ M)
⑥ M = M − wi
⑦ P = P + Pi
⑧ else Break;
⑨ if (M > 0)
⑩ P = P + Pi $(\frac{M}{wi})$

} O(n)

$T(n) = O(n) + O(n \log n) + O(n)$

$T(n) \Rightarrow O(n \log n)$

Greedy Knapsack Problem :→

m=15

obj	1	2	3	4	5	6	7
P	10	5	15	7	6	18	3
w	2	3	5	7	1	4	1

∴ Method 1 (Profit Based) :→

3 (obj 7)
5 (obj 2)
6 (obj 5)
7 (obj 4)
10 (obj 1)
15 (obj 3)
18 (obj 6)

15 − 4 = 11
11 − 5 = 6
6 − 2 = 4

$1 \times 18 + \frac{11}{5} \times 15 + \frac{6}{2} \times 10 + \frac{4}{7} \times 7$

= 18 + 33 + 30 + 4

⇒ 85

∴ Method 2 (Min weight) :→

object 4 (7)
object 3 (5)
object 6 (4)
object 2 (3)
object 1 (2)
object 7 (1)
object 5 (1)

$1 \times 6 + \frac{14}{1} \times 3 + \frac{11}{2} \times 10 + \frac{1}{3} \times 5$

= 6 + 42 + 55 + 1.7

⇒ 104.7

∴ Method 3 :→

P/w	6	1.7	3	1	6	4.5	3

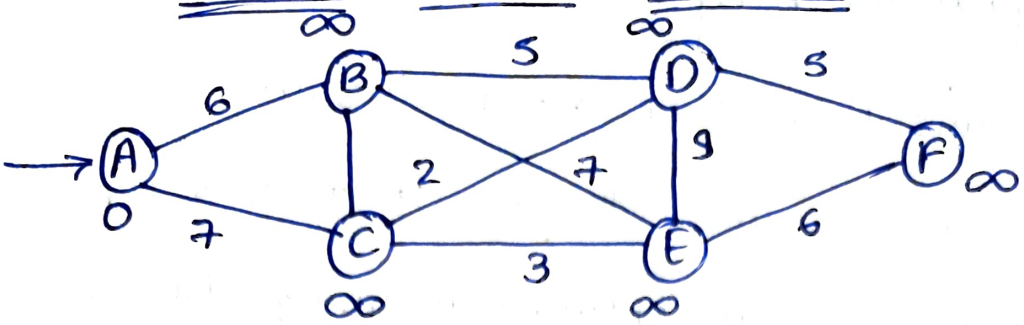
$= 6 \times 1 + \frac{14}{2} \times 10 + \frac{12}{7} \times 7 + \frac{5}{5} \times 15$

= 6 + 70 + 12 + 15

⇒ 103

obj 6 (4)
obj 1 (2)
obj 5 (1)

Implement Dijkstra Algorithm



Relax Function

if $d(u) + w(u,v) < d(v)$

$$d(v) = d(u) + w(u,v)$$

A	B	C	D	E	F
0	~~∞~~	~~∞~~	∞	∞	∞
A	6	7	~~∞~~	~~∞~~	∞
A, B, C	6	7	9	10	~~∞~~
A, B, C, D, E	6	7	9	10	14
A, B, C, D, E, F	6	7	9	10	14

DYNAMIC PROGRAMMING :→

- It is mainly an optimisation over recursion.
- wherever we see a recursion solution that has repeated calls for the same input we can optimize it using dynamic programming.
- The idea is to simply store the result of sub-problem that we do not have to recompute when needed later.

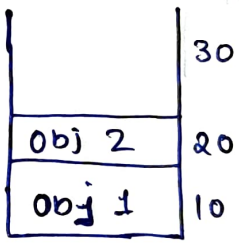
→ There are various type of DP :→

→ 0/1 Knapsack Problem
→ longest common Seaquence (LCS)
→ Matrix Chain Multiplication
→ All pair Shortest path (Flyod Warshal Algo)

0/1 Knapsack Problem :→

M=50

Obj	1	2	3
P	60	100	120
W	10	20	30
P/W	6	5	4



Disadvantage : In this, fraction is not used

Object	1	2	3	4
Profit	1	4	5	7
weight	1	3	4	5

W=7

$$KS(n,w) = \begin{cases} 0, & n=0 \text{ or } w=0 \\ KS(n-1,w), & wt[n] > w \end{cases}$$

$$\max \begin{cases} KS(n-1, w-wt[n]+P[n]) \\ KS(n-1, w) \end{cases}$$

Total $(n+1)(w+1)$

	wt(n)	P	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7
n=0	0	0	0	0	0	0	0	0	0	0
n=1	1	1	0	1	1	1	1	1	1	1
n=2	3	4	0	1	1	4	5	5	5	5
n=3	4	5	0	1	1	4	5	6	6	9
n=4	5	7	0	1	1	4	5	7	8	9

$KS(0,0) = 0$

$Ks(1,1) = (0,0) + 1 = 0+1 = 1 = Ks(0,1) \Rightarrow \boxed{1}$

$Ks(1,2) = \max \begin{cases} Ks(0,2)+1) = 0+1 \Rightarrow \boxed{1} \\ Ks(0,2) \end{cases}$

$Ks(3,7) = Ks(2,3) + 5 = 4+5 \Rightarrow \boxed{9}$

obj	1	2	3	4
	0	1	1	0

D-5 $\quad 4 - 4 = 0$

$T(n) = (n+1)(w+1)$

$\boxed{T(n) = O(nw)}$

Largest common Subsequence (LCS) :→

Strings : ABCDE

Substrings : A, B, BC, CDE [CE, BE] ✗

Subsequence :→ ∀ Strings → Subsequence

AB, BC, BCE {CB, DB} ✗

X → ABCDMF
Y → AEDFK
} Longest common Subsequence : |ADF| = 3

|AD| = 2
|DF| = 2

* LCS (m,n) { 0 if m=0 or n=0
1 + LCS (m-1, n-1) if X[m] = Y[n]
max {LCS(n, m-1), LCS(m-1, n)} if X[m] ≠ Y[n]

Best Case

X → ABCD

Y → ABCD

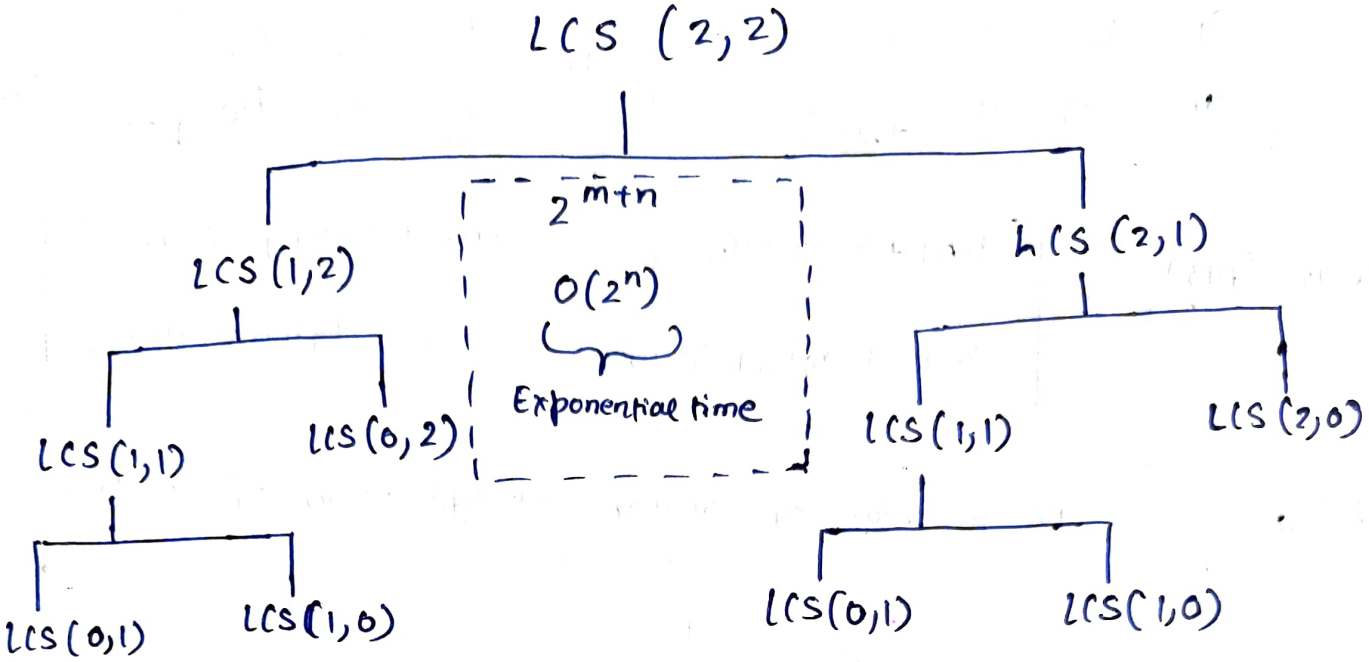
⇒ 1 + LCS (3,3)

⇒ 1 + LCS (2,2)

⇒ 1 + LCS (1,1)

⇒ 1 + LCS (0,0)

Recurrence Tree :→



Advantage (DP) :→

if it [LCS (1,1)] calculate once then also other calculated which means they save time, memory.

Bottom-up Method

x → ABCD (m)

Y → AEDF (n)

[(m+1) (n+1)]

AD

T(n) = O(m,n)

N \ M	0	A	B	C	D
0	0	0	0	0	0
A	0	1	1	1	1
E	0	1	1	1	1
D	0	1	1	1	2
F	0	1	1	1	2

Backtrack

Matrix Chain Multiplication (MCM) :→

$$A=\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \end{bmatrix}_{2\times 3}, \quad B=\begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \\ b_5 & b_6 \end{bmatrix}_{3\times 2}$$

$$(AB)_{2\times 2}=\begin{bmatrix} a_1b_1+a_2b_3+a_3b_5 & a_1b_2+a_2b_4+a_3b_6 \\ a_4b_1+a_5b_3+a_6b_5 & a_4b_2+a_5b_4+a_6b_5 \end{bmatrix}$$

$(AB)_{2\times 2\times 3} = 12$ → No. of Scalar Multiplication of matrix
$\quad i\ j\ k$

(*)

ABC

A(BC) (AB)C

* $A_{2\times 5}$ $B_{5\times 10}$ $C_{10\times 3}$

$B_{5\times 10}$ $C_{10\times 3}$

$= 5\times 3\times 10 = 150$

$A_{2\times 5}$ $(BC)_{5\times 3}$

$= 2\times 3\times 5 = 30+150$

$\Rightarrow 180$

$A_{2\times 5}$ $B_{5\times 10}$

$= 2\times 10\times 5 \Rightarrow 100$

$(AB)_{2\times 10}$ $C_{10\times 3}$

$= 100 + (2\times 3\times 10)$

$\Rightarrow 160$

(*) $A \mid_{K=1} (B\ C)$ $\overset{m_1}{\ }$ $\overset{m_2}{\ }$ $\overset{m_3}{\ }$ $\Rightarrow (AB) \mid_{K=2} C$

Recursive Method :→

$\therefore A_{2\times 5}$ $B_{5\times 10}$ $C_{10\times 3}$

$P_0\times P_1$ $P_1\times P_2$ $P_2\times P_3$

$$M = \text{Min} \begin{cases} M[i,K] + m(K+1,j) + P_{i-1} P_K P_j & \text{if } i<j \text{ ; Range } (i \le K < j) \\ 0 & \text{if } i=j \end{cases}$$

$(BC)_{5\times3} = B_{5\times10} \quad C_{10\times3}$

$= 5\times3\times10 \Rightarrow 150$

By using formula

A(BC) :→

$M[2,3] = m[2,2] + m[3\times3] + 5\times10\times3 \Rightarrow \boxed{150}$

$A|BC = m[1,1] + m[2,3] + 2\times5\times3 \Rightarrow 30$

$\Rightarrow \boxed{180}$

(AB)C :→

$(AB)_{2\times10} = A_{2\times5} \quad B_{5\times3} = 100$

$M[1,2] = m[1,1] + m[2,2] + 2\times5\times10 \Rightarrow 100$

$\Rightarrow \boxed{160}$

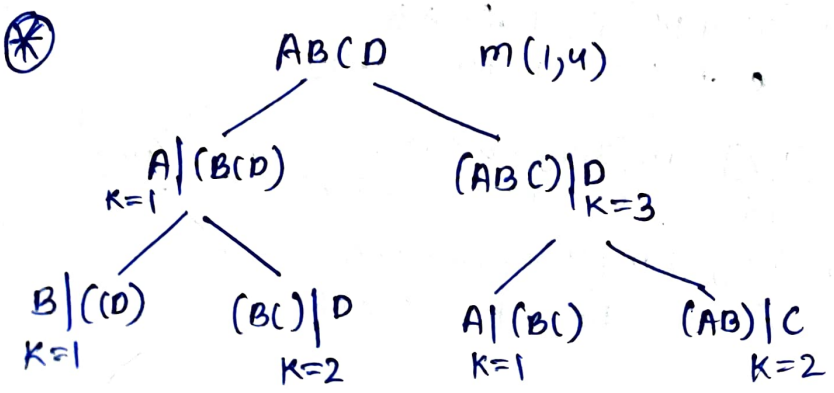
$M[1,3] = m[1,2] + m[3,3] + 2\times10\times3$

$= 100 + 0 + 60$

$\boxed{\Rightarrow 160}$

$(ABC) = M[1,3] = \min \begin{cases} M[1,1] + m[2,3] + 150 = 180 \\ M[1,2] + M[3,3] + 160 = 160 \end{cases}$

(*)



Matrix Sequences :→

$\langle 2, 1, 3, 4, 5 \rangle$

$A_{2\times1}$ (m_1) $B_{1\times3}$ (m_2) $C_{3\times4}$ (m_3) $D_{4\times5}$ (m_4)

M =

	1	2	3	4
1	0	6	20	42
2		0	12	32
3			0	60
4				0

K =

	1	2	3	4
1	0	1	1	1
2		0	2	3
3			0	3
4				0

$M[1,2] = m[1,1] + m[2,2] + 2\times3\times1 = 6$

$m[2,3] = m[2,2] + m[3,3] + 1\times4\times3 = 12$

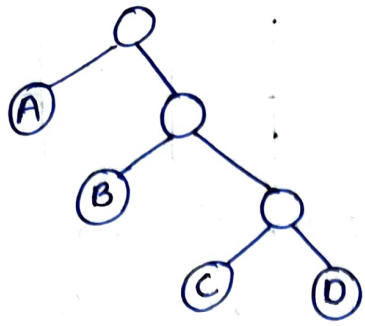
$m[3,4] = m[3,3] + m[4,4] + 3\times5\times4 = 60$

$m[1,3] = \min \begin{cases} m(1,1) + m(2,3) + 2\times1\times4 \\ \quad 0 + 12 + 8 \Rightarrow 20 \\ m(1,2) + m(3,3) + 2\times3\times4 \\ \quad 6 + 0 + 24 \Rightarrow 30 \end{cases}$ → min (20)

$m[2,4] = \min \begin{cases} \min[2,2] + \min[3,4] + 1\times3\times5 \\ \quad 0 + 60 + 15 \Rightarrow 75 \\ \min[2,3] + \min[4,4] + 1\times4\times5 \\ \quad 12 + 0 + 20 \Rightarrow 32 \end{cases}$ → min (32)

$M[1,4] = \min \begin{cases} m[1,1] + m[2,4] + 2\times1\times5 \\ \quad 0 + 32 + 10 \Rightarrow 42 \\ m[1,2] + m[3,4] + 2\times3\times5 \\ \quad 6 + 60 + 30 \Rightarrow 76 \\ m[1,3] + m[4,4] + 2\times4\times5 \\ \quad 20 + 0 + 40 \Rightarrow 60 \end{cases}$ → min (42)

(*) A | B | (CD)
K=1 K=2



Time complexity :→

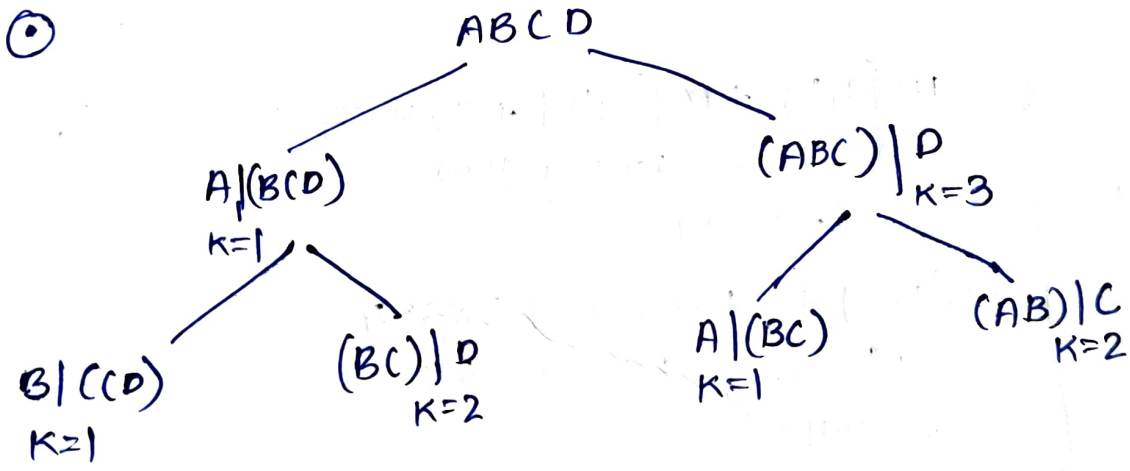
(A(B(CD)))

$T(n) = O(n^3)$

Q 4 Matrices

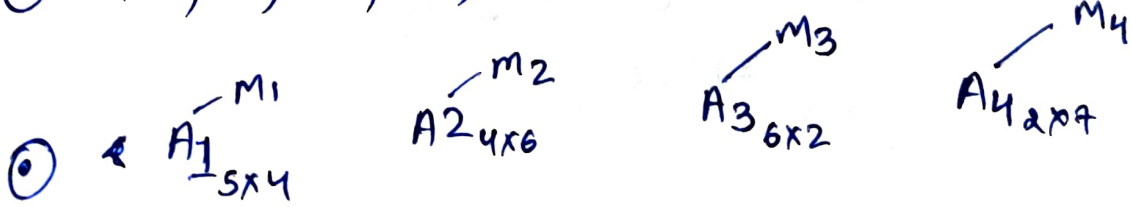
$\langle A_1, A_2, A_3, A_4 \rangle$

$A1_{5\times4}$ $A2_{4\times6}$ $A3_{6\times2}$ $A4_{2\times7}$



P_0 P_1 P_2 P_3 P_4

$\langle 5, 4, 6, 2, 7 \rangle$



M =

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

,

K =

	1	2	3	4
1	0	1	1	4
2		0	2	3
3			0	3
4				0

$m[1,2] = m[1,1] + m[2,2] + 5\times4\times6 = 120$

$m[2,3] = m[2,2] + m[3,3] + 4\times6\times2 = 48$

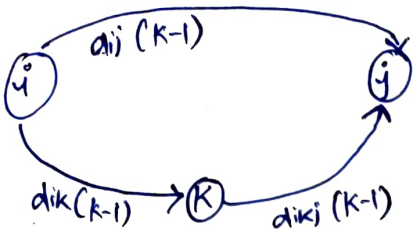
$m[3,4] = m[3,3] + m[4,4] + 6\times2\times7 = 84$

$m[1,3] = \min \begin{cases} m[1,1] + m[2,3] + 5\times4\times2 \\ 0 + 48 + 40 \Rightarrow 88 \\ m[1,2] + m[3,3] + 5\times6\times2 \\ 120 + 0 + 60 \Rightarrow 180 \end{cases}$ min (88)

$m[2,4] = \min \begin{cases} m[2,2] + m[3,4] + 4\times6\times7 \\ 0 + 84 + 168 \Rightarrow 252 \\ m[2,3] + m[4,4] + 4\times2\times7 \\ 48 + 0 + 56 \Rightarrow 104 \end{cases}$ min (104)

$m(1,4) = \min \begin{cases} m[1,1] + m[2,4] + 5\times4\times7 \\ 0 + 104 + 140 \\ \Rightarrow 244 \\ m[1,2] + m[3,4] + 5\times6\times7 \\ 120 + 84 + 210 \\ \Rightarrow 414 \\ m[1,3] + m[4,4] + 5\times2\times7 \\ 88 + 0 + 70 \\ \Rightarrow 158 \end{cases}$

All Path Shortest path (Floyd-Warshal Algo) :→



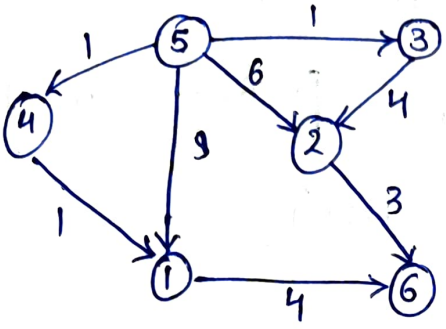
$d^{(0)}_{(5,6)} = \infty$

$d^{(1)}_{(5,6)} = 13\ (5,1,6)$

$d^{(2)}_{(5,6)} = 9\ (5,2,6)$

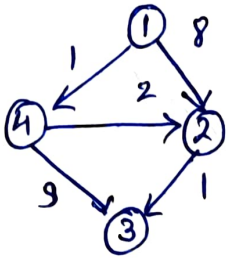
$d^{(3)}_{(5,6)} = 8\ (5,3,2,6)$

$d^{(4)}_{(5,6)} = 6\ (5,4,1,6)$



Min Shortest path = $d^{(4)}_{(5,6)}$

Q



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array}\begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} \infty & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ \infty & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{array} \Rightarrow d(0)$$

$$d(1) = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ \infty & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix}, \quad d(2) = \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ \infty & \infty & 0 & \infty \\ \infty & 2 & 3 & 0 \end{bmatrix}$$

$$d(3) = \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ \infty & \infty & 0 & \infty \\ \infty & 2 & 3 & 0 \end{bmatrix}, \quad d(4) = \begin{bmatrix} 0 & 3 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ \infty & \infty & 0 & \infty \\ \infty & 2 & 3 & 0 \end{bmatrix}$$

FLOYD – WARSHAL ALGO (W)

1- $n \leftarrow row(n)$

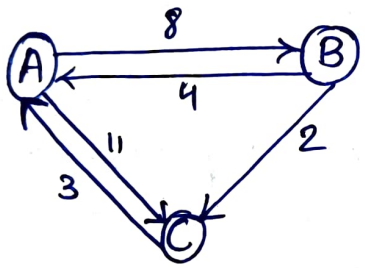
2- $D^{(0)} \leftarrow W$

3- for $K \leftarrow 1$ to $n \longrightarrow n$

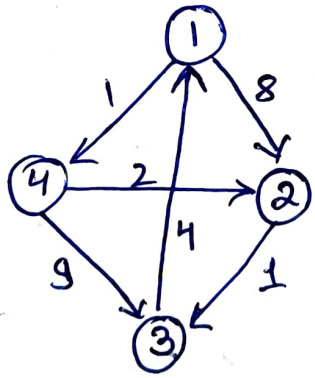
4- do for $i = 1$ to $n \longrightarrow n^2$ $= n + n^2 + n^3$

5- do for $j = 1$ to $n \longrightarrow n^3$ $\boxed{T(n) = O(n^3)}$

6- $d_{ij}^{(K)} \leftarrow \min\left(d_{ij}^{(K-1)}, d_{ik}^{(K-1)} + d_{kj}^{(K-1)}\right)$

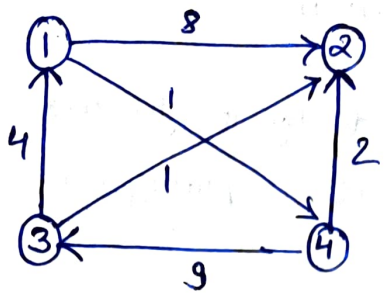


②



$$d(4) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{array}$$

Q



$$d(0) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{array}$$

$$d(1) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{array}$$

This, MCM, Knapsack %

$$d(2) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 8 & (9) & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & (3) & 0 \end{bmatrix} \end{array}$$

$$d(3) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 8 & 9 & 1 \\ (5) & 0 & 1 & (6) \\ 4 & 12 & 0 & 5 \\ (7) & 2 & 3 & 0 \end{bmatrix} \end{array}$$

$$d(4) = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & (3) & (4) & 1 \\ 5 & 0 & 1 & 0 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{array}$$

DYNAMIC PROGRAMMING

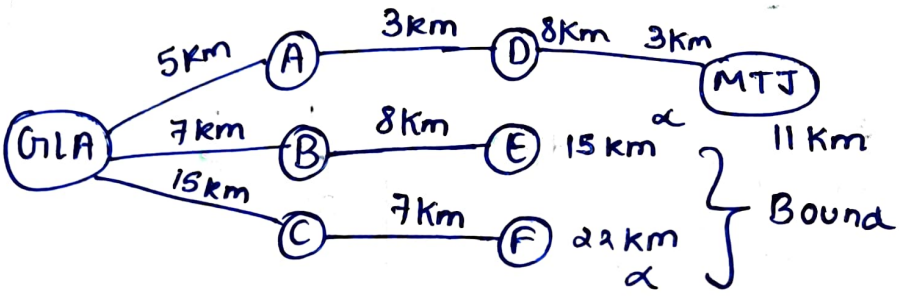
- Top-Down Approach
  - → Memoization + Recursion
- Bottom-Up Approach
  - → Tabulation

Branch and Bound Algorithm :→

for find optimal solution is used by it for combintory, discrete and general mathematical optimization.

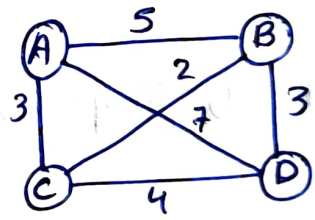
Branching: It is the process of generating sub-problem

Bounding: It refers to ignore partial solution that can't be better then best solution.



TSP (Travelling Salesman Problem) :→

∴ Move with min. cost



Eg:→

$A \overset{5}{-} B \overset{3}{-} C \overset{4}{-} D \overset{3}{-} A$

$\Rightarrow 15$

$A \overset{7}{-} D \overset{3}{-} B \overset{2}{-} C \overset{3}{-} A$

$\Rightarrow 15$

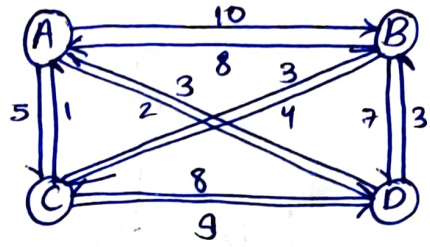
$A \overset{3}{-} C \overset{2}{-} B \overset{3}{-} D \overset{7}{-} A$

$\Rightarrow 15$

$A \overset{7}{-} D \overset{4}{-} C \overset{2}{-} B \overset{5}{-} A$

$\Rightarrow 18$

Q



Weight Matrix:

	A	B	C	D
A	∞	10	5	3
B	8	∞	9	7
C	1	6	∞	9
D	2	3	8	∞

Row Reduction:

	A	B	C	D
A	∞	7	2	0
B	1	∞	2	0
C	0	5	∞	8
D	0	1	6	∞

Coloumn Reduction:

	A	B	C	D
A	∞	6	0	0
B	1	∞	0	0
C	0	4	∞	8
D	0	0	4	∞

$\Rightarrow M_1$ (parent)

TR = 13 + 3

Total Reduction (TR) = 16

$(A^{row(\infty)} - B^{col(\infty)})$

	A	B	C	D	
A	∞	∞	∞	∞	0
B	∞	∞	0	0	0
C	0	∞	∞	8	0
D	0	∞	4	8	0

(Complete row/column ∞)

Aleast one zero

Reduction = 0

TC = Parent Matrix + R + (A-B)

= 16 + 0 + 6 = 22

(A-C)

	A	B	C	D	
A	∞	∞	∞	∞	
B	1	∞	∞	0	
C	∞	4	∞	8	4
D	0	0	∞	∞	

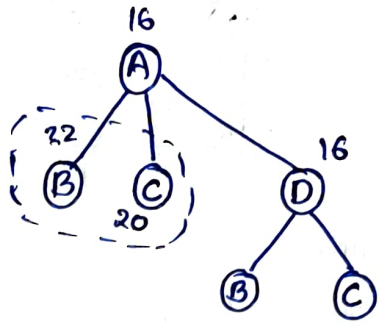
$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 0 \\ \infty & 0 & \infty & 4 \\ 0 & 0 & \infty & \infty \end{bmatrix}$$

TC = 16 + 4 + 0 => 20

(A-D)

	A	B	C	D
A	∞	∞	∞	∞
B	1	∞	0	∞
C	0	4	∞	∞
D	∞	0	4	∞

R=0



TC = 16 + 0 + 0 = 16

(A-D-B)

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	0	∞
C	0	∞	∞	∞
D	∞	∞	∞	∞

R=0

TC = 16 + 0 + 0 = 16

(A-D-C)

	A	B	C	D	
A	∞	∞	∞	∞	
B	1	∞	∞	∞	1
C	∞	4	∞	∞	4
D	∞	∞	∞	∞	5

⇒

∞	∞	∞	∞
∞	∞	∞	∞
∞	∞	∞	∞
∞	∞	∞	∞

TC = 16 + 5 + 4 = 25

A-D-B-C

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	∞
D	∞	∞	∞	∞

R=0

TC = 16 + 0 + 0
= 16

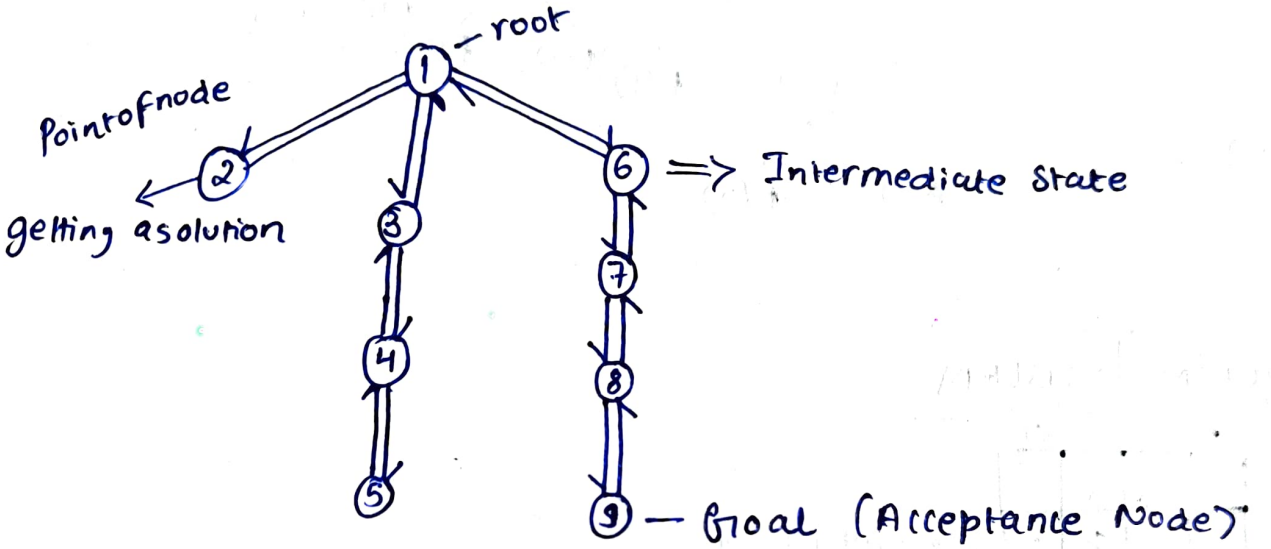
A-D-B-C-A
3 + 3 + 9 + 1 ⇒ 16

Time complexity = O(n!)

BACKTRACKING

* Backtracking is a problem solving technique that involves finding a solution infinitely in different options

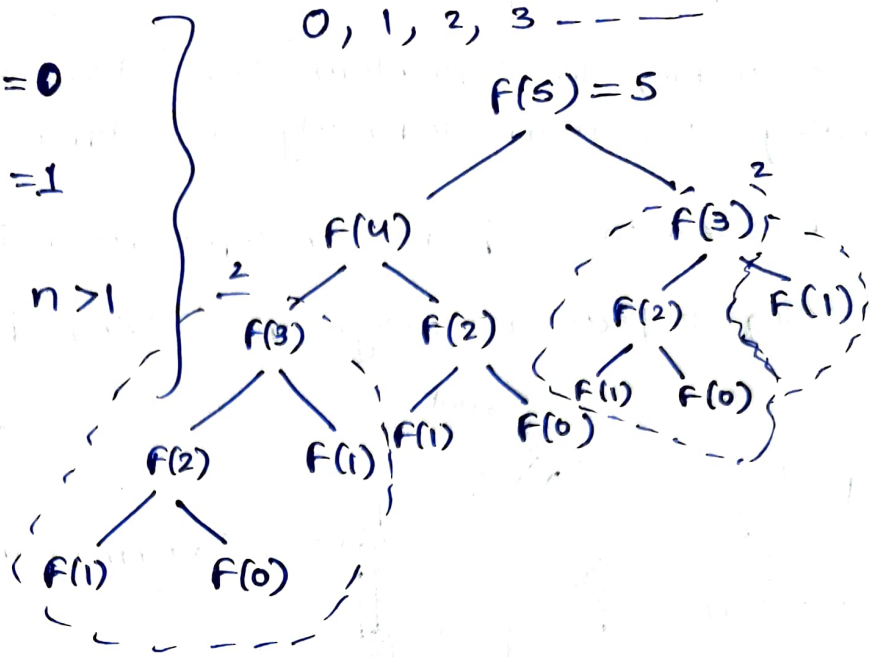
* It is used to explore multiple possibility in problem such as finding a path in maze or solving puzzle.



Q State Comparison Among recursive Method, Greedy Method, Dynamic Approach, Branch and Bound Method, Backtracking Method with Eg.

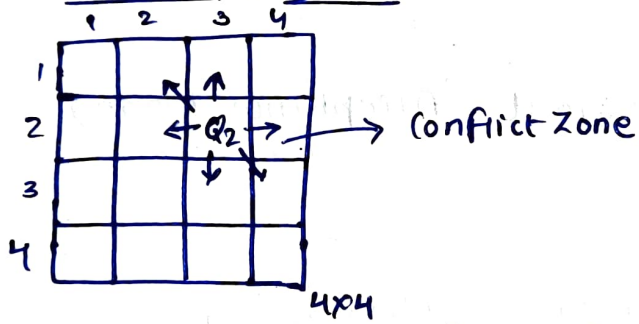
fib(5)

$$f(n) = \begin{cases} 0, & \text{if } n=0 \\ 1, & \text{if } n=1 \\ f(n-1)+f(n-2) & n>1 \end{cases}$$



$O(2^n)$

N-Queen Problem



Q(2,3) = 5 (row, column)

Q(1,4) = 5

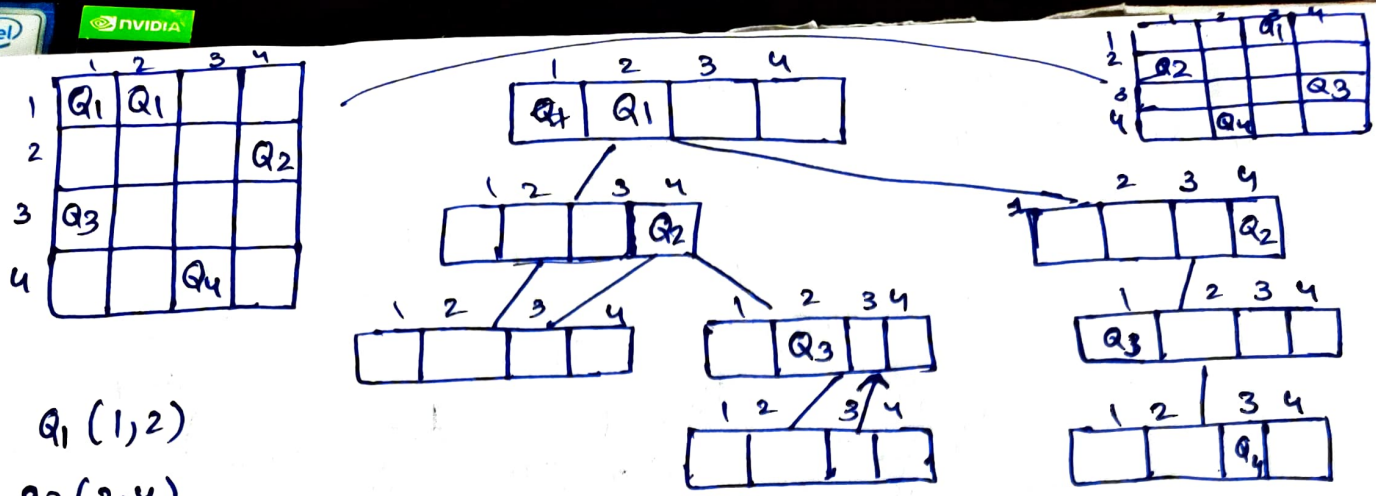
Q(3,2) = 5

Conflict Zone

Q(1,2) = 1-2 = -1

Q(3,4) = 3-4 = -1

Note:→ After Addition or Subtraction of row and column we should not have 5 and -1 as result as they will create conflicts.



$Q_1(1,2)$

$Q_2(2,4)$

$Q_3(3,1)$

$Q_4(4,3)$

Time complexity = $n=4$

$(n-1)(n-2)(n-1)$

$= 4 \times 3 \times 2 \times 1 = 24$

Time complexity = $O(n!)$

Sum of Subsets Problem

Set $t = \{3, 5, 7, 6, 3\}$

$i=0$ $j=2$

Sum target = 2

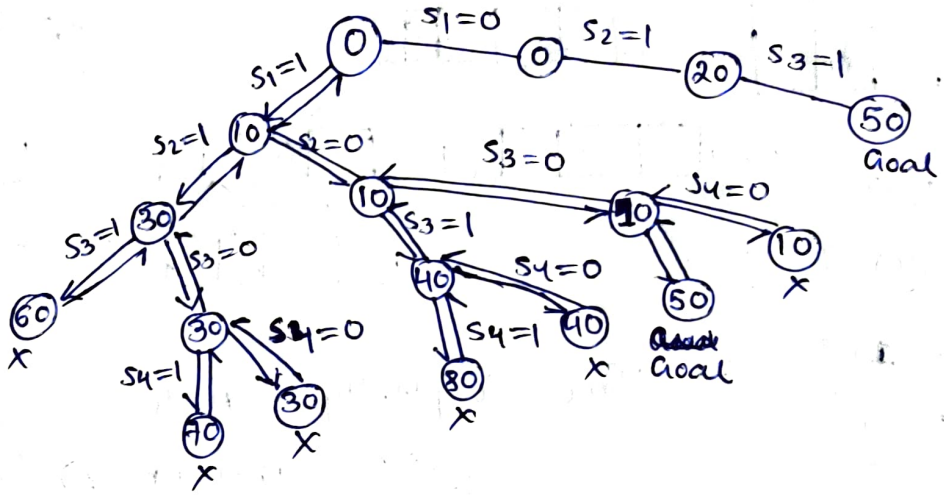
$7+6=13$

$2 \times 2 \times 2 \times 2 \times 2 = 2^5$

Suppose n elements $= 2^n$

Time complexity = $O(2^n)$

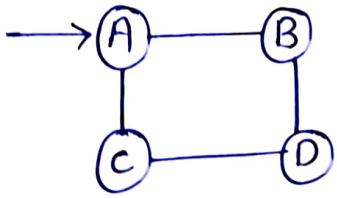
$n = \{10, 20, 30, 40\}$ target Sum = 50.
$s_1 \; s_2 \; s_3 \; s_4$



Time Complexity = $O(2^n)$

$2^4 = 16$

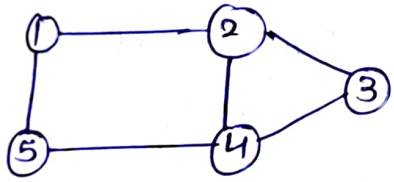
Hamiltonian Cycle :→

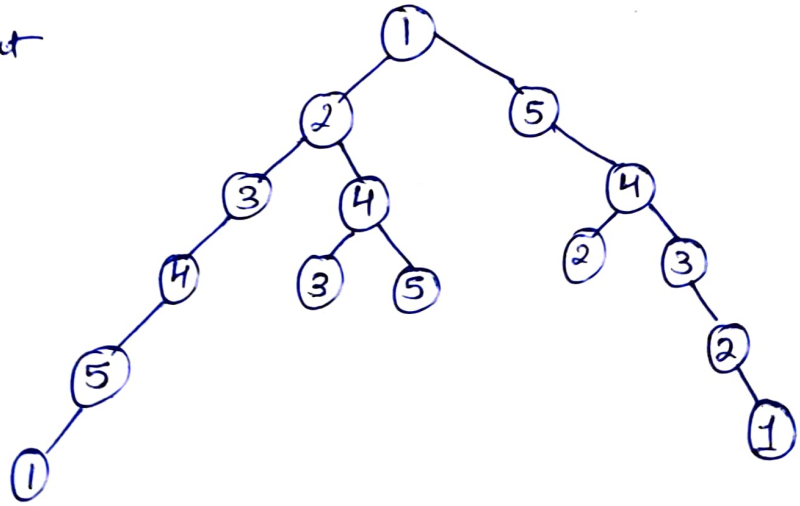


A - (B - D - C) - A

No - repeat

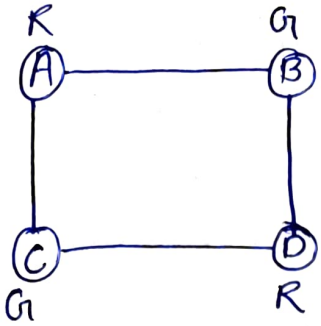
Q





$T(n) = O(n!)$

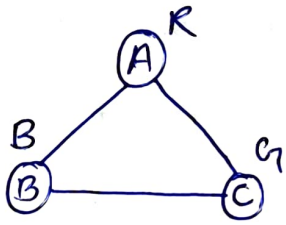
Graph Coloring Problem

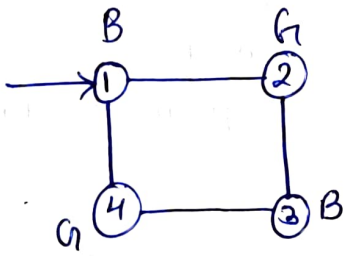


⇒ Chromatic number = k=2

→ fixed no. of colour are used

→ Connected vertex or Adjacent vertex should not be same colour.

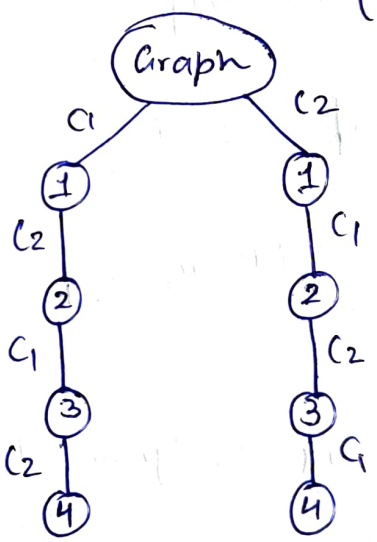




SST (State Space Tree)

Solution Space Tree

(DFS)

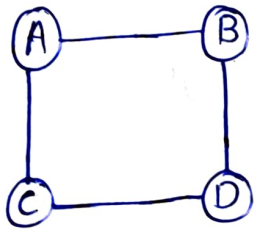


number of Solution = 1×2 = 2

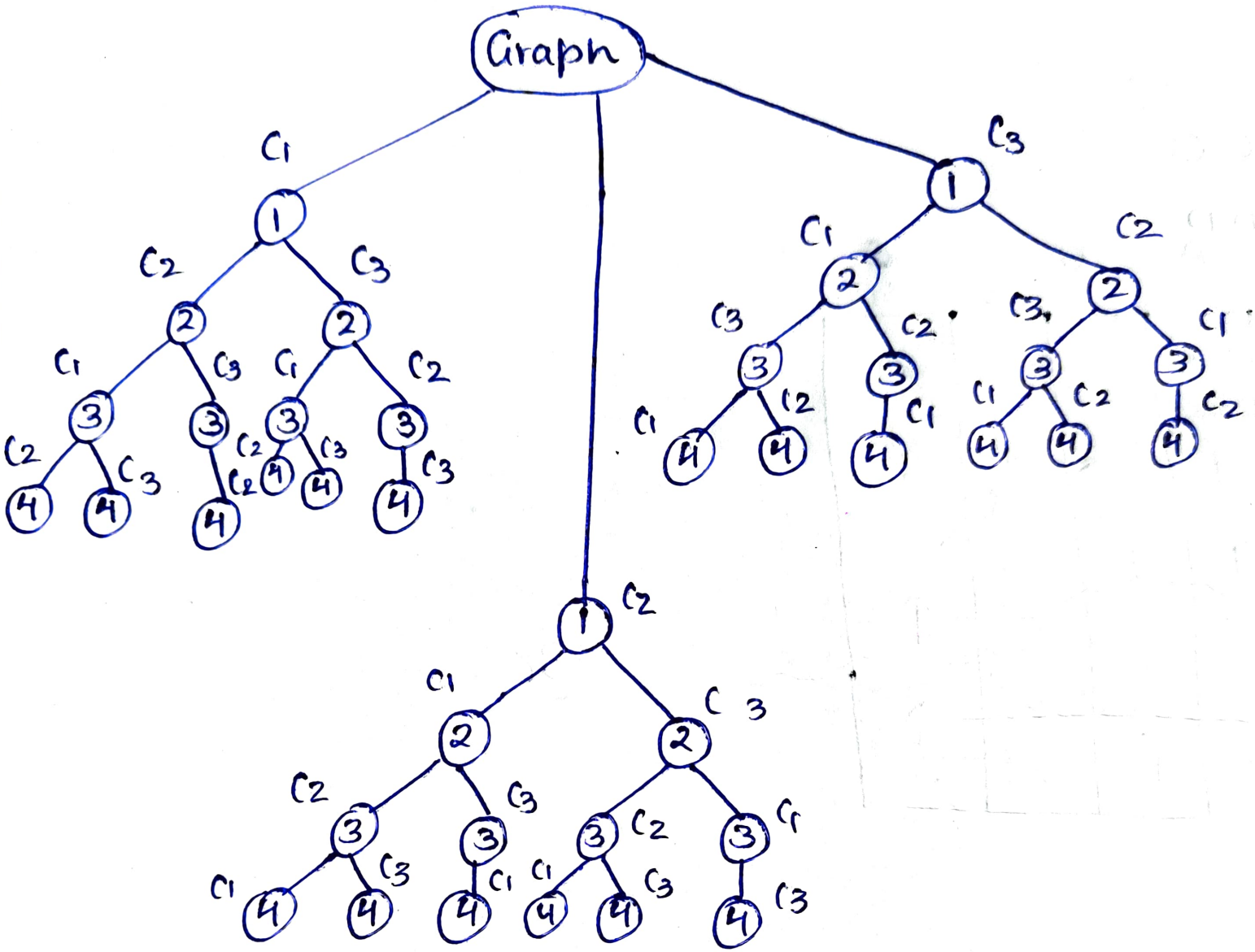
time complexity = $(m^n V)$

↓ no. of colour

Q

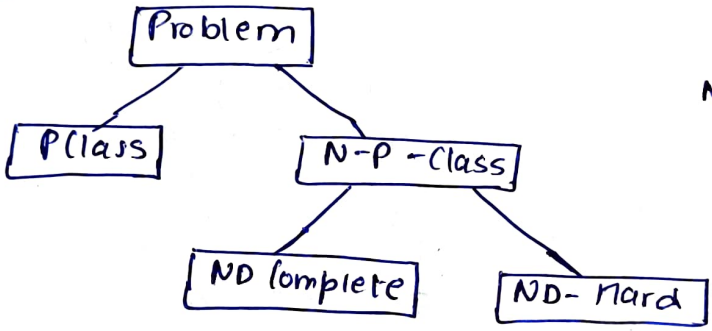


colour (R, G, B)


Graph

Computational Complexity Classes :→

- In his field of algo understands how efficient an algo to solve a kruskal algo.
- This is where complexity classes come into place.
- Complexity classes group problem based on resources (Time and Space) required to solve here.
- In this simple term, they help us how understand how a problem, what kind of algo can solve is.



P: Polynomial
NDP: Non- Deterministing Polynomial Time

CLASS P PROBLEM (Polynomial Time) :→

→ Problem in P class are those that can be solved by an Algo in polynomial class

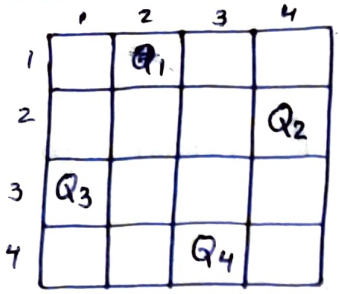
→ In other words, time taken to solve these problems grows at a polynomial rate as the input size increase.

Those we can solve easily in computer:
Bubble Sort, Merge Sort, Insertion Sort, DFS, BFS etc.

→ Problem in NP class those where a solution are verified polynomial time but we do not nessary know how to solve them efficiently.

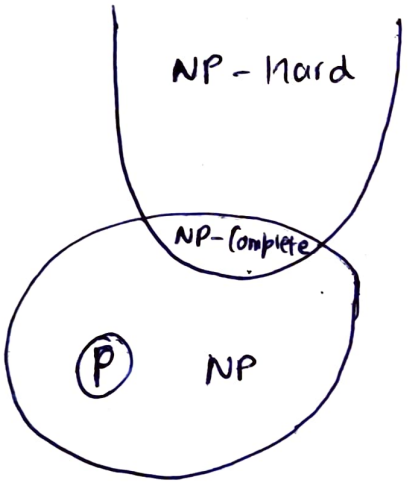
This problem can be solved by Exponential time

Real Time Verification :→



⇒ n!

Example :→ Travelling Salesman Problem (n!)
Hamiliton Path Problem (n!)
Graph Coloring Problem (Vm^V)



NP- Hard :→

A Problem NP hard if every problem is can be polynomially reduced to it.

Example :→ TSP ; Graph Coloring Problem ;
0/1 Knapsack problem

NP Complete :→

A Problem is NP Complete if a problem it is in NP hard and NP. $\forall NPC \longrightarrow NPH$

Example :→ TSP ; Hamiltonian Path ; Sum of Subset problem ; Graph Coloring.

* Some problem is a NP hard but not NP Complete.

Example →

→ During Halting Problem

→ Past Correspondence Problem

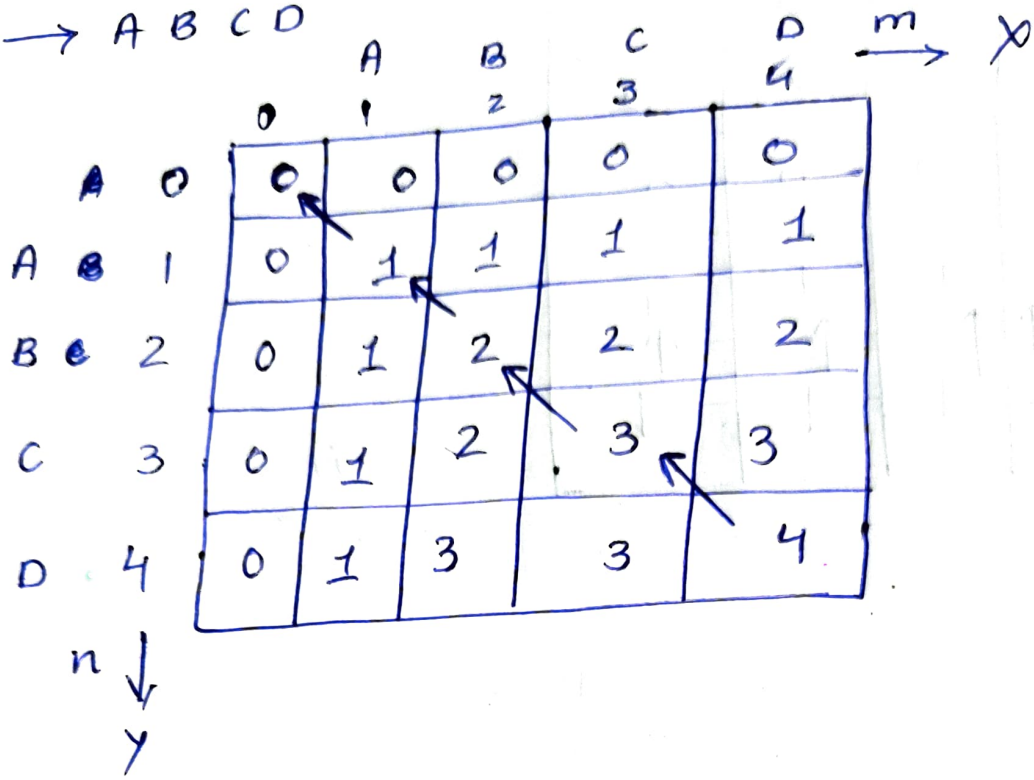
→ Integer problem

→ Min - Circuit Size Problem.

LCS: →

X → A B C D

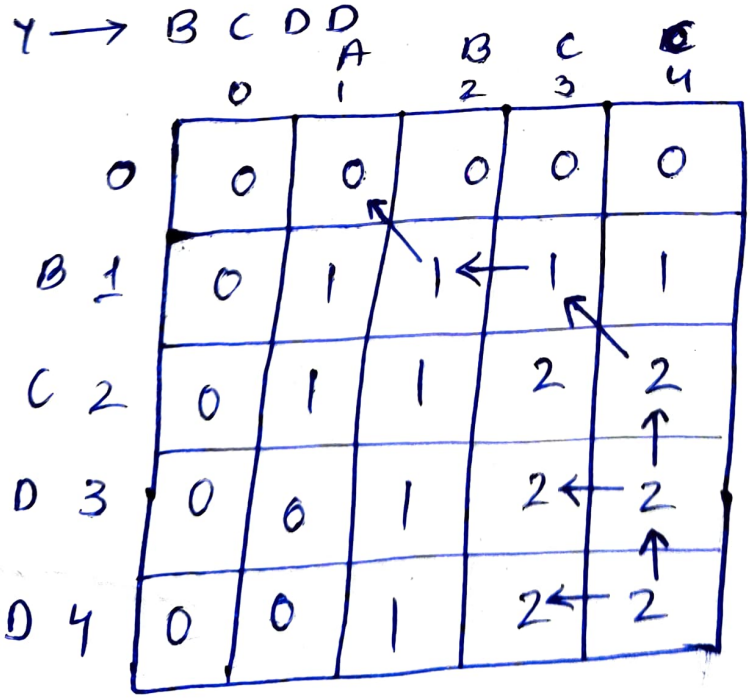
Y → A B C D



ABCD

Y → A B C C

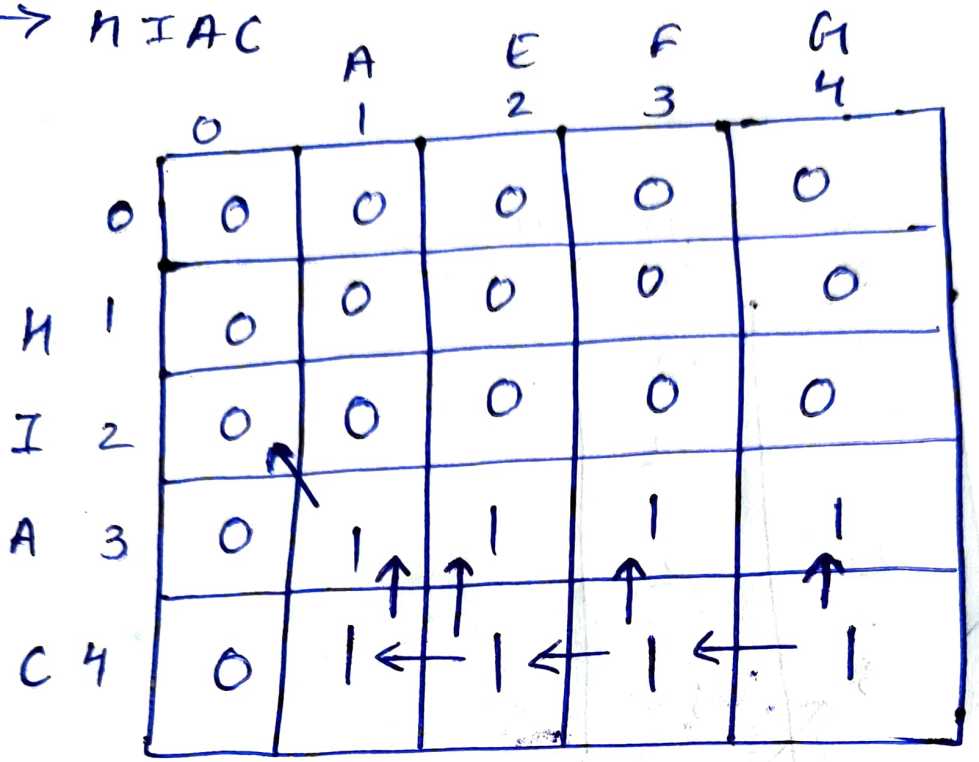
Y → B C D D



BC

X → AEFG

Y → HIAC



A